



HpC: A Calculus for Hybrid and Mobile Systems

XIONG XU*, Institute of Software Chinese Academy of Sciences, China

JEAN-PIERRE TALPIN*, INRIA, France

SHULING WANG, Institute of Software Chinese Academy of Sciences, China

HAO WU, Institute of Software Chinese Academy of Sciences, China

BOHUA ZHAN, Huawei Technologies Co., Ltd., China

XINXIN LIU, Institute of Software Chinese Academy of Sciences, China

NAIJUN ZHAN[†], Peking University & Zhongguancun Laboratory, China

Networked cybernetic and physical systems of the Internet of Things (IoT) immerse civilian and industrial infrastructures into an interconnected and dynamic web of hybrid and mobile devices. The key feature of such systems is the hybrid and tight coupling of mobile and pervasive discrete communications in a continuously evolving environment (discrete computations with predominant continuous dynamics). In the aim of ensuring the correctness and reliability of such heterogeneous infrastructures, we introduce the hybrid π -calculus (**HpC**), to formally capture both mobility, pervasiveness and hybridisation in infrastructures where the network topology and its communicating entities evolve continuously in the physical world. The π -calculus proposed by Robin Milner et al. is a process calculus that can model mobile communications and computations in a very elegant manner. The **HpC** we propose is a conservative extension of the classical π -calculus, i.e., the extension is “minimal”, and yet describes mobility, time and physics of systems, while allowing to lift all theoretical results (e.g. bisimulation) to the context of that extension. We showcase the **HpC** by considering a realistic handover protocol among mobile devices.

CCS Concepts: • **Theory of computation** → **Process calculi**; **Timed and hybrid models**; *Algebraic language theory*; *Operational semantics*.

Additional Key Words and Phrases: π -calculus, Hybrid Systems, Internet of Things, Bisimulation

ACM Reference Format:

Xiong Xu, Jean-Pierre Talpin, Shuling Wang, Hao Wu, Bohua Zhan, Xinxin Liu, and Naijun Zhan. 2025. HpC: A Calculus for Hybrid and Mobile Systems. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 121 (April 2025), 26 pages. <https://doi.org/10.1145/3720478>

1 Introduction

Today’s Internet of Things (IoT) consists of networked cybernetic and physical systems (CPSs) that improve coordination and observation of large infrastructures by sensing and actuating the

*Equal contributors.

[†]Corresponding author.

Authors’ Contact Information: **Xiong Xu**, National Key Laboratory of Space Integrated Information System, Institute of Software Chinese Academy of Sciences, Beijing, China, xux@ios.ac.cn; **Jean-Pierre Talpin**, INRIA, Rennes, France, jean-pierre.talpin@inria.fr; **Shuling Wang**, National Key Laboratory of Space Integrated Information System, Institute of Software Chinese Academy of Sciences, Beijing, China, wangsl@ios.ac.cn; **Hao Wu**, KLSS (CAS) and SKLCS, Institute of Software Chinese Academy of Sciences, Beijing, China, wuhao@ios.ac.cn; **Bohua Zhan**, Huawei Technologies Co., Ltd., Beijing, China, bzhan@ios.ac.cn; **Xinxin Liu**, KLSS (CAS) and SKLCS, Institute of Software Chinese Academy of Sciences, Beijing, China, xinxin@ios.ac.cn; **Naijun Zhan**, School of Computer Science and Key Laboratory of High Confidence Software Technology, Peking University & Zhongguancun Laboratory, Beijing, China, znj@ios.ac.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/4-ART121

<https://doi.org/10.1145/3720478>

environment using increasingly complex distributed protocols. Most of such systems consist of heterogeneous endpoints, physical or cybernetic, of highly variable capacities, resources, connectivity, mobility, and security. These endpoints combine hardware-embedded functionalities in complex, distributed, discrete/continuous control loops between small, autonomous and mobile endpoints and large, secure, high-performance servers, often so from individual, sporadic or intermittent interactions between “end-user apps”, e.g., smartphones, vehicles, and large infrastructures, e.g., clouds, traffic or satellite networks. When designing networked and mobile CPSs of the IoT, we need to take both the effect of the dynamic change of the network topology (mobility) and the correctness of the hybrid behaviours (continuous evolution, discrete computation and control, and their fusion) of the communicating entities within the network into account. We call such systems *Hybrid and Mobile Systems*, of which the complexity reflects in the following four aspects.

Mobility The physical location as well as the communication link of an entity within the network keeps changing, i.e., the communication topology is dynamically changed as private client-server channels are transferred as messages using servicing channels.

Privacy The communication within the network usually has privacy requirements. Some entities may form a local group to communicate with each other, but other joining the network may not share with this group.

Openness In the runtime of the network, new entities and communication links may be created and, meanwhile, old or obsolete ones may be deleted at any time, i.e., the network is an open and dynamic system.

Hybridisation Physical and cybernetic entities in the network perform hybrid behaviour, coupling discrete sensing and actuation protocols and topologically pervasive continuous physical behaviours.

Hybrid and mobile systems are usually safety-critical, for instance, internet-connected vehicles, next generation train networks, air-traffic control, unmanned autonomous systems. Any defect in the design of protocols controlling such distributed systems may lead to catastrophic consequences. Assuring the safety of heterogeneity systems with mobility and dynamicity forms a challenge which must be addressed with formal guarantees as mandated by many certification standards for safety-critical system design. Dealing with the tightly-coupled combination of mobility, privacy, openness, and hybridisation is the most difficult aspect of this challenge. Unfortunately, and to the best of our knowledge, no existing calculus covers the full spectrum and complexity of such infrastructures, despite their commonality in civil and industrial networks.

To address this issue, this article presents a conservative extension of the classical π -calculus, the hybrid π -calculus (**H π C**). In the π -calculus, proposed by Robin Milner et al., channels, variables, and data are all first-class citizens: they are called names. This makes a fundamental difference between the π -calculus and other process calculi such as CSP (Communicating Sequential Processes) or CCS (Calculus of Communicating Systems). A name can be used as a channel to transfer other names and meanwhile it can be treated as a message transferred by names: a first-class citizen. More details about π -calculus can be found in [34–37]. The convention that “names are first-class citizens” makes the π -calculus powerful in characterising mobility. Therefore, we adopt this convention in the **H π C**, where a name can uniformly be used as a channel, a parameter, an ordinary, continuous or physical variable. Thanks to this convention, the **H π C** can model complex networked and mobile CPSs in a very natural and elegant manner, just like the classic π -calculus for discrete mobile communication systems alone.

Contributions. This paper presents the first characterisation of a π -calculus that has the capability to compositionally orchestrate the interaction of discretely timed processes and continuously timed

ordinary differential equations (ODEs). The **HpC** provides a hybrid calculus in which all objects are first-class citizens:

- Discrete channels and continuous variables are treated uniformly;
- Discrete and continuous processes are equal hybrid processes;
- All channels and processes, discrete or continuous, have both meanings in the discrete and continuous operational semantics; and
- Last but not least, the **HpC** is the first calculus to support control-theoretic approximate bisimulation in the presence of dynamic and mobile channels and processes.

This combination forms a process algebra, the **HpC**, to describe protocols amongst hybrid and mobile systems by allowing for the specification of mobility, real-timed continuous evolution and discretely-timed concurrent computations and communications. The lean and simple embedding of discrete and continuous functions in the π -calculus (there is one proof system for discrete interaction and one for continuous evolution) allows us to define a theory of bisimulation applying to discrete and continuous processes alike, providing the most expressive model for the verification of hybrid and mobile systems. While perhaps surprising, this result is in turn similar to, and an extension of, earlier results in [39], which proved the discrete relative completeness of differential dynamic logic, and in [55], which proved the discrete relative completeness of hybrid Hoare logic. We introduce the real-world case study of a handover protocol for a train control system to demonstrate the benefits of the **HpC** to provide an expressive, sound and precise modelling and verification workflow to model protocols dominated by physics, distribution, real time, dynamicity and mobility, along with a theory of bisimulation to verify them.

Paper Organisation. After reviewing related works in Section 2, some motivating examples in Section 3 give an overview of the **HpC**, whose formal syntax and operational semantics are specified in Section 4 and Section 5. Then, a theory of strong, weak, and approximate bisimulation for the **HpC** are introduced in Section 6. The case study of a handover protocol of a real-world train control system is demonstrated in Section 7. Section 8 concludes.

2 Related Works

An important research field and corpus of related works addresses the modelling and verification of combined continuous and discrete dynamics, also called hybrid systems. It can be roughly classified into the following categories:

- Graphical modelling tools with simulation engines (Simulink [32, 33], Ptolemy, Modelica), allow to model closed-loop systems using diagrams, verify and validate them by simulation.
- Automata-based modelling and model-checking based on reachable sets computation abstractly model systems using hybrid automata [1, 22]. Hybrid automata extend finite-state automata by attaching ordinary differential equations (ODEs) to model continuous evolution in each state of the system. Verification is conducted by computing reachable sets possibly with abstraction and approximation [12, 16, 17, 26];
- Process algebra-like modelling together with theorem proving, by which a system is modelled as a process-algebraic term, and verification is done by theorem proving, e.g., differential dynamical logic [38] and hybrid Hoare logic [30, 55].
- Extended Event-B framework, based on refinement and automated proof to model and verify hybrid systems, for instance, core hybrid Event-B [5–7] and Event-B hybridisation [15].

Unlike the aforementioned approaches, advanced concurrency models such as the π -calculus [34] and its variants do support the expression of mobility and dynamic communication topology. With these process-algebraic methods, verification is conducted using several forms of bisimulation such

as strong, weak, and branching bisimulation [37, 48]. None of the aforementioned approaches to the modelling and verification of hybrid systems address mobility and dynamicity.

2.1 Process Algebras for Timed and Hybrid Systems

The notion of Hybrid Process Algebra (HyPA) was first proposed in [14] and it is an extension of the Algebra of Communicating Processes (ACP) to model hybrid systems. HyPA processes were then linearised in [47], where a corresponding simulation tool was provided. Hybrid χ [45] is an extension of the simulation language χ to model hybrid systems. Hybrid χ combines the theories of process algebra and hybrid automata and can model the dynamic behaviour of systems. Reference [46] defined the methods that transform hybrid χ to other formal models that describe hybrid systems. The work of [9] combined the process algebra with timing [4], propositional signals [3], and a process algebra that can describe the interleaving of discrete actions and continuous evolution.

A process algebra combining time and mobility, named TiMo (Timed Mobility), was proposed in [13] and it was then extended to real-time systems as rTiMo [2]. However, rTiMo only supports local communication, while in the real-world application, entities can communicate remotely. Thus, a new model BigrTiMo [51], the combination of Bigraph model and rTiMo, was present. Compared to the current calculus for mobile distributed systems, BigrTiMo can not only capture mobility, but also describe space and time constraints. Recently, a timed calculus with mobility, called MTCWN [50], was proposed to capture mobility in wireless networks.

Of inspiration to the definition of the **HpC** is He's works on Hybrid CSP (HCSP) [21, 56] which provide a uniform characterisation of discrete processes and continuous behaviours in Hoare's CSP [23]. However, it does not support mobility, i.e., the dynamic introduction and transmission of channel names as first-class objects. The classical π -calculus supports mobility that way, but does depict the continuous evolution of CPSs. The Φ -calculus, the first extension of π -calculus to model hybrid systems, was proposed in [43], and a corresponding verification framework based on spatial logic was given in [42]. However, the issue of mobility is not the concern of the calculus, and the tight coupling of hybrid behaviour and mobile communication is also not addressed.

The session π -calculus [44] is an extension of the π -calculus that can model communication protocols between processes in a more structured manner. The calculus enjoys a powerful multiparty session type system by which properties like deadlock-freedom and liveness can be proved. However, it cannot model continuous behaviour. Another calculus of sessions is the multiparty motion session calculus [31]. It is equipped with continuous-time motion primitives to describe affine approximations of continuous agents behaviours. It serves as a modelling language for programming and schedule the coordination of multiple robots. However, it does not support general forms of ODEs or mobility, i.e., the topology of communication never changes.

2.2 Process Algebras for the Internet of Things

There are some works based on process calculi for the modelling and verification of IoT. For example, the IoT-calculus [28] is the first process calculus for IoT. It can capture some basic features of IoT networks, such as the partial topology of communications and the interaction between sensors, actuators and computing processes to provide useful services to the end user. IoT-LySa [10] is a process calculus endowed with a static analysis that tracks the provenance and the route of IoT data, and detects how they affect the behaviour of smart objects. However, neither IoT-calculus nor IoT-LySa can describe timed behaviour. A process calculus of IoT systems, called CaIT, is proposed in [29], where timed behaviours, with desirable time, consistency and fairness properties, are supported. Recently, a secure mobile real-time process calculus for specifying and reasoning about IoT systems, called SMrCaIT [11], was proposed. It supports not only value-passing communication but also name-passing communication, and additionally, it can strictly separate process actions and

mobility modelling by providing parametric mobility models. However, SMrCaIT cannot describe the continuous evolution of entities in physical world.

2.3 Verification of Hybrid Systems

A lot of existing works on verification of hybrid systems use reachability-analysis and theorem proving methods. Most of the works on reachability analysis model hybrid systems use hybrid automata, which are verified by computing reachable sets [1]. On the theorem proving side, differential dynamic logic [38, 40] reasons about hybrid systems using an extension of dynamic logic to include continuous evolution. HCSP [21] is an extension of CSP that includes continuous evolution and interruption by communication. Proof systems based on Hoare logic have been proposed for HCSP [20, 30, 55]. In this paper, we conduct the verification of **HpC** processes based on the notion of bisimulation, which covers the behaviour of discrete communication, mobility of names, dynamicity of processes, and of continuously timed functions.

There have been some work on simulation and approximate simulation relations between hybrid systems. Due to the presence of ODEs, it is not possible to maintain an exact simulation relation between hybrid systems or between a hybrid system and a discrete program executable on a computer. Hence, the concept of approximate bisimulation is proposed [19] to allow errors up to a bounded distance between states of the two systems. The distance between two states is usually defined as the maximum of Euclidean distances of values of variables under states, which is actually one special case of the quantitative bisimulation proposed in [18, 27]. The work [24] proposes the use of two parameters on both observable states and transition labels as precisions. Using a similar notion of approximate bisimulation with two parameters, the work [54] proved correctness of code generation from HCSP models into SystemC [54] and then into C [49], in the sense that the original model and the discrete code are approximately bisimilar.

3 Overview: Motivating Examples

We introduce some preliminary examples to illustrate how continuous behaviours are represented in the **HpC**, starting with continuous time, and then highlight more of its features using more elaborate ones.

$$\text{BigBen} \triangleq \{0 \mid \dot{c} = 1, \{\bar{c}\}\}$$

The process named **BigBen** has a free name c denoting the clock (highlighted in blue for clarity). Intuitively, the implementation of **BigBen** only interacts with the outside world using the (discrete) communication channel c . The evolution of **BigBen** is continuously defined by an ordinary differential equation (ODE) $\dot{c} = 1$ with the initial condition 0, where $\dot{c}$ is the abbreviation of the derivative $\frac{dc}{dt}$ and $\dot{c} = 1$ means c increases linearly (with respect to time). The set $\{\bar{c}\}$ defines the *interface* of the ODE. The notation $\bar{c}$ is an output action, indicating that the real-time value of the clock c can be sensed (read) at any time during its evolution. However, the clock of **BigBen** cannot be actuated (written). This would have to be specified by an additional input action c in the interface, i.e., $\{\bar{c}, c\}$, allowing one to reset **BigBen** with a new value of c . **BigBen** can serve as a global clock that can be sensed everywhere in the scope of c . We can, on the contrary, use c as a private name to model the side effect of a timing constraint, as follows:

$$\text{wait}(d) \triangleq (\nu c)\{0 \mid \dot{c} = 1 \ \& \ c < d\}$$

Process **wait** is parameterised by d , to denote the duration and the scope of c (private and localised by ν). No one in the environment can read or write its value (the interface of the ODE is empty). The constraint $c < d$ means that the evolution of the ODE terminates once the value of c reaches d . It is called the *boundary condition* of the ODE. Concretely, one may depict the evolution

of process `wait(d)` by the following transition:

$$\text{wait}(d).P \xrightarrow{d} P$$

In the reminder, we will reuse process `wait` to express time delays. Now, let us consider a process observing time from `BigBen` repeatedly. The process named `Observer` recursively (modality μ) reads time t from channel `c`, performs some time-sensitive task `Task(t)` and then waits for two time units before scheduling the next run. The `Observer` can interact with `BigBen` periodically by sharing channel `c`. This interaction can be modelled by the parallel composition `BigBen || Observer`.

$$\text{Observer} \triangleq \mu x. c(t). \text{Task}(t). \text{wait}(2). \bar{x}$$

REMARK 1. Notice that name `c` in `BigBen` is a physical variable of the ODE $\dot{c} = 1$. Meanwhile, it is used as an output channel (a sensor) in its analog-digital interface $\{\bar{c}\}$ with process `Observer`. The clock `c` evolves continuously according to $\dot{c} = 1$. During its evolution, `BigBen` can provide a real-time value to the observer using the discrete output channel $\bar{c}$, whenever it is requested to. In this sense, `c` could be called a “continuous channel”. In the **HpC**, however, we do not distinguish between physical variables and channels. They are all “first-class citizens” following the convention of the π -calculus.

3.1 Bouncing Ball

Now, let us consider a classic: a ball falling from an initial height of 5m with initial velocity 0m/s. The ball falls with a dynamics given by two ODEs $\dot{h} = v$, its height, and $\dot{v} = -9.8$ m/s², its velocity. The behaviour of the ball is modelled by the following process:

$$\text{Ball} \triangleq \left\{ \begin{array}{l|l} 5 & \dot{h} = v \\ 0 & \dot{v} = -9.8 \end{array} \right\}, \{v, \bar{v}\}$$

The interface $\{v, \bar{v}\}$ indicates that the environment can sense ($\bar{v}$) and actuate (v) the velocity of the ball. Once the ball hits the ground, its velocity becomes discontinuous: it is reversed. We assume that the collision of the ball with the ground is inelastic and that its velocity is reversed by a factor of -0.8 , i.e., the ball bounces back and loses 0.2 energy. The effect of hitting the ground implies:

$$\text{Ground} \triangleq \mu x. (vc) \{0 \mid \dot{c} = 1 \ \& \ h > 0 \vee v \geq 0\}. v(v_0). \bar{v}(-0.8v_0). \bar{x}$$

where `c` is the local time on the ground. It means that the ground waits until the elevation h of the ball reaches 0 ($h \leq 0$) with negative velocity ($v < 0$), i.e., the boundary condition $h > 0 \vee v \geq 0$ becomes **false**. When that happens, it reads the velocity of the ball by the input action $v(v_0)$. The discrete variable v_0 stores the value of the velocity and the ground inverts it to $-0.8 \times v_0$ by performing the output action $\bar{v}(-0.8v_0)$. The bouncing ball system can be defined by the parallel composition `Ball || Ground`, which causes a Zeno-behaviour: it generates an infinite execution but the sum of the durations of all the timed actions in the execution is bounded by a constant.

3.2 Mobile Vehicle

At last, we introduce an example reflecting the capability of the **HpC** in modelling mobility, i.e., the migration of channel and physical variable names as messages through servicing channels between processes. We consider a vehicle moving back and forth between two base stations, which control the movement of the vehicle alternately. The behaviour of the `Vehicle` can be modelled as follows. Modality v creates names p , v , and a to denote its position, velocity, and acceleration, respectively. These names are private to the `Vehicle`, which then sends them using the shared channel b_1 to

notify base station **Base₁** that it may sense and actuate the **Vehicle** using these names.

$$\mathbf{Vehicle} \triangleq (vp, v, a)\overline{b_1}\langle p, v, a \rangle. \left\{ \begin{array}{l|l} 0 & \dot{p} = v \\ 0 & \dot{v} = a \\ 0 & \dot{a} = 0 \end{array} \right\} \{ \overline{p}, \overline{v}, a \}$$

Once synchronised, the vehicle behaves according to the set of ODEs $\{\dot{p} = v, \dot{v} = a, \dot{a} = 0\}$, where p , v , and a are set to 0 initially. The set $\{\overline{p}, \overline{v}, a\}$ defines the interface of the ODEs: the output actions $\overline{p}$ and $\overline{v}$ denote that the real-time values of position (p) and velocity (v) of the vehicle can be sensed (read); the input action a indicates that the value of the acceleration (a) of the vehicle can be actuated (written) by the environment interacting with the ODEs.

$$\mathbf{Base}_1 \triangleq !b_1(p, v, a).p(p_0). \left(\begin{array}{l} [p_0 < 100].v(v_0).\overline{a}\langle f_1(p_0, v_0) \rangle.\mathbf{wait}(1).\overline{b_1}\langle p, v, a \rangle \\ + \\ [p_0 \geq 100].\overline{b_2}\langle p, v, a \rangle \end{array} \right)$$

Notice that input actions p and v are not in the interface, indicating that the position (p) and velocity (v) of the vehicle cannot be modified by the environment. The behaviour of base station b_1 uses the operator $+$ to denote a non-deterministic choice between two conducts and replication ($!$) to model mutual recursion (between stations 1 and 2). Replication $!P$ is equivalent to $P||!P$. Intuitively, it is the parallel composition of infinite P s. When an instance of P is forked, it receives the actual parameters (p, v, a) on channel b_1 and then forks a process to read the real-time position p_0 of the vehicle by its name p , i.e., $p(p_0)$. If the vehicle is still within the domain of the base station ($p_0 < 100$), then (1) the base station continues to read the velocity by its name v , i.e., $v(v_0)$, (2) adjust the acceleration (a) according to the sensed velocity (v_0) and position (p_0) of the vehicle by an algorithm f_1 , making the vehicle move forwards, i.e., $\overline{a}\langle f_1(p_0, v_0) \rangle$, (3) wait for 1 second, i.e., $\mathbf{wait}(1)$, and (4) repeats the procedure by rebooting itself, i.e., $\overline{b_1}\langle p, v, a \rangle$. Otherwise, if the vehicle has moved out of range of the station ($p_0 \geq 100$), the station contacts its neighbour to take control over the vehicle by sending it the physical variables (p , v , and a) of the vehicle over channel b_2 . **Base₁** will then wait until being invoked again by the other station **Base₂** over channel b_1 .

Symmetrically, the behaviour of the second base station is modelled by the matching replication. Function f_2 orders the vehicle to move back to the first station until the position of the vehicle reduces to 10. When that happens, the first base station is invoked by $\overline{b_1}\langle p, v, a \rangle$ to take control back on the vehicle again. The system can be modelled using parallel composition $\mathbf{Vehicle}||\mathbf{Base}_1||\mathbf{Base}_2$, allowing the vehicle to move back and forth between the two base stations.

$$\mathbf{Base}_2 \triangleq !b_2(p, v, a).p(p_0). \left(\begin{array}{l} [p_0 > 10].v(v_0).\overline{a}\langle f_2(p_0, v_0) \rangle.\mathbf{wait}(2).\overline{b_2}\langle p, v, a \rangle \\ + \\ [p_0 \leq 10].\overline{b_1}\langle p, v, a \rangle \end{array} \right)$$

4 The Hybrid π -Calculus

In this section, we introduce the syntax of the **HpC**, along with the notations and the structural congruence laws that will be used throughout the paper.

4.1 Names and Expressions

We assume an enumerable set of *names* $\mathcal{N}$, e.g. $x, y, z, \dots$, possibly with superscript or subscript. Unlike discrete channels, which carry values, continuous physical variables are designated by a subset of names $\mathcal{V} \subseteq \mathcal{N}$ to denote functions from time to values, usually noted v, u, w , etc. Expressions e are defined by the grammar $e ::= c \mid x \mid \text{op } \vec{e}$, where c is a constant such as a real

or a string, $x \in \mathcal{N}$ is a name, op represents an operator, and $\vec{e} ::= \emptyset \mid e, \vec{e}$ is a tuple or vector of such expressions, and especially, $\vec{c}$ denotes a vector of constants. We use $|\vec{e}|$ and $\mathcal{N}(\vec{e})$ to denote the length of and the set of names in $\vec{e}$, respectively. A Boolean expression B can be defined (or derived) from the strictly smaller grammar $B ::= \text{false} \mid e = e \mid e < e \mid B \wedge B \mid \neg B$.

4.2 Syntax

The syntax of **HpC**, defined below, is an extension to the classic π -calculus in [34, 35] with two constructs, highlighted in **bold**: (1) a general form of *guard* $[B]$, and (2) prefix $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}$ to describe the *continuous* behaviour of a physical process.

In the remainder and for clarity, we use terms **P**, **Q**, and **R** to represent processes and **M**, **N**, and **W** to denote sums $\sum_{i \in I} \pi_i. \mathbf{P}_i$, where sub-terms guard a process $\mathbf{P}_i$ by a prefix π_i , which can be empty (silence τ), an input $x(\vec{y})$, an output $\bar{x}(\vec{e})$, a Boolean clause $[B]$, or a continuous $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}$. The empty sum ($I = \emptyset$) stands for *inaction* and is noted as process $\emptyset$. The restriction of a name x to the scope of **P** is noted $(\nu x)\mathbf{P}$, its replication noted $!\mathbf{P}$ and its parallel composition to $\mathbf{P}'$ noted $\mathbf{P} \parallel \mathbf{P}'$.

The term $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).\mathbf{P}$ denotes a continuous process. From left to right, it consists of a vector of expressions $\vec{e}_0$ representing the initial state of the ODE's variables $\vec{v}$ and $\vec{e}$ its *vector field* which we require to be locally Lipschitz-continuous (as formally defined in Theorem 6.7). The Boolean condition B defines the domain constraints (range or boundary) of the continuous variable. Once B is violated by the state of the ODE, e.g. $\vec{c}$, then its evolution stops immediately and its continuation $(\vec{y}).\mathbf{P}$ is applied to the final state $\vec{c}$, reducing to $\mathbf{P}\{\vec{c}/\vec{y}\}$. Finally, the set $\mathcal{R} \subseteq \{v, \bar{v} \mid v \in \mathcal{N}(\vec{v})\}$ is called the *ready set* or *interface* of the ODE. It declares the channel names which can be sensed or actuated by discrete processes. For brevity, we omit B if it is **true** and omit $\mathcal{R}$ if it is $\emptyset$.

Definition 4.1 (Syntax).

$\pi ::= \tau$	(Silent)	$\mathbf{P} ::= \sum_{i \in I} \pi_i. \mathbf{P}_i$	(Sum)
$\mid x(\vec{y})$	(Input)	$\mid (\nu x)\mathbf{P}$	(Restriction)
$\mid \bar{x}(\vec{e})$	(Output)	$\mid \mathbf{P} \parallel \mathbf{P}'$	(Parallel)
$\mid [B]$	(Guard)	$\mid !\mathbf{P}$	(Replication)
$\mid \{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y})$	(Continuous)		

REMARK 2. For a continuous action $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}$, i.e., we do not assume that $\mathcal{N}(\vec{e}) \subseteq \mathcal{N}(\vec{v})$ or $\mathcal{N}(B) \subseteq \mathcal{N}(\vec{v})$ (as opposed to autonomous systems, whose continuous variables are local). For instance, $\{0 \mid \dot{v} = w \& u < 1, \{v, \bar{v}\}\}$ allows to reference external physical variables w and u .

4.3 Scope of Names

We write $\mathcal{B}(\mathbf{P})$ for the names bound in $\mathbf{P}$ and conversely, $\mathcal{F}(\mathbf{P})$ for the *free* names of $\mathbf{P}$. For example, $\mathcal{F}(x(\vec{y}).\mathbf{P}) = \{x\} \cup \mathcal{F}(\mathbf{P}) - \mathcal{N}(\vec{y})$ and $\mathcal{B}(x(\vec{y}).\mathbf{P}) = \mathcal{B}(\mathbf{P}) \cup \mathcal{N}(\vec{y})$. We adopt Barendregt's Convention that the free and bound names of each process are disjoint: $\mathcal{F}(\mathbf{P}) \cap \mathcal{B}(\mathbf{P}) = \emptyset$ for all $\mathbf{P}$ and hence $\mathcal{N}(\mathbf{P}) = \mathcal{F}(\mathbf{P}) \uplus \mathcal{B}(\mathbf{P})$.

4.4 Structural Congruence

Structural congruence $\equiv$ is defined as the smallest congruence satisfying the following laws:

- If $\mathbf{P}$ is an α -conversion of $\mathbf{Q}$, then $\mathbf{P} \equiv \mathbf{Q}$.
- Commutativity: $\mathbf{P} \parallel \mathbf{Q} \equiv \mathbf{Q} \parallel \mathbf{P}$ and $\mathbf{M} + \mathbf{N} \equiv \mathbf{N} + \mathbf{M}$.
- Associativity: $(\mathbf{P} \parallel \mathbf{Q}) \parallel \mathbf{R} \equiv \mathbf{P} \parallel (\mathbf{Q} \parallel \mathbf{R})$ and $(\mathbf{M} + \mathbf{N}) + \mathbf{W} \equiv \mathbf{M} + (\mathbf{N} + \mathbf{W})$.

Equivalences of $\mathbf{P} \parallel \emptyset \equiv \mathbf{P}$, $\mathbf{M} + \emptyset \equiv \mathbf{M}$, $(\nu x)\emptyset \equiv \emptyset$, $(\nu x)(\mathbf{P} \parallel \mathbf{Q}) \equiv \mathbf{P} \parallel (\nu x)\mathbf{Q}$ if $x \notin \mathcal{F}(\mathbf{P})$, and so on are proved with the strong bisimulation relation $\sim$ defined in Section 6.1.

5 Operational Semantics

In this section, we first define the substitutions on expressions and processes in Section 5.1. Then, we define states and flows in Section 5.2 and 5.3 to describe the dynamics of a physical process. A state maps names to reals while a flow is a function over time describing the trajectory or evolution of a physical process. Section 5.4 specifies the operational semantics of the proposed **HpC**, where we can see how continuous variables are viewed as channels sampled by discrete processes.

5.1 Substitutions

A substitution $\Delta \triangleq \{e_1/x_1, \dots, e_n/x_n\}$, also written $\{e_1, \dots, e_n/x_1, \dots, x_n\}$ or $\{\vec{e}/\vec{x}\}$, maps pairwise distinct names x_i (i.e. $i \neq j \Rightarrow x_i \neq x_j$) of a process **P** (resp. an expression e) to an expression $x_i\Delta = e_i$, for all $1 \leq i \leq n$, written $\mathbf{P}\Delta$ (resp. $e\Delta$), and acts as the identity otherwise.

5.2 States

A state $\sigma_V : V \rightarrow \mathbb{R}$ maps the continuous variables in $V \subseteq \mathcal{V}$ to reals. We note it σ if its domain $\text{dom}(\sigma) = V$ is specified. The union of two states $\sigma \cup \sigma'$ is *defined* iff. σ and σ' agree on common names: for all $x \in \text{dom}(\sigma) \cap \text{dom}(\sigma')$, $\sigma(x) = \sigma'(x)$. Let $\vec{\omega}$ be a vector of names and constants. The evaluation of $\vec{e}$ in state σ , denoted $\llbracket \vec{e} \rrbracket_\sigma$, is defined by the following rules:

$$\frac{}{\llbracket c \rrbracket_\sigma = c} \quad \frac{x \in \text{dom}(\sigma)}{\llbracket x \rrbracket_\sigma = \sigma(x)} \quad \frac{x \notin \text{dom}(\sigma)}{\llbracket x \rrbracket_\sigma = x} \quad \frac{\text{op } \llbracket \vec{e} \rrbracket_\sigma = \vec{\omega}}{\llbracket \text{op } \vec{e} \rrbracket_\sigma = \vec{\omega}} \quad \frac{}{\llbracket e, \vec{e} \rrbracket_\sigma = \llbracket e \rrbracket_\sigma, \llbracket \vec{e} \rrbracket_\sigma}$$

For brevity, $\llbracket \vec{e} \rrbracket$ denotes $\llbracket \vec{e} \rrbracket_{\sigma_0}$, for example, $\llbracket 1+2 \rrbracket = 3$, $\llbracket x \rrbracket = x$, and $\llbracket x = x \rrbracket = \text{true}$, while $\llbracket 1/0 \rrbracket$, $\llbracket x+1 \rrbracket$, and $\llbracket x > y \rrbracket$ are undefined. Evaluation acts as substitution, hence $\llbracket x \rrbracket_\sigma = x$ if $x \notin \text{dom}(\sigma)$.

5.3 Flows

A flow $\rho_V^d : [0, d) \rightarrow V \rightarrow \mathbb{R}$ is defined by a function over time using a set of continuous variables $V \subseteq \mathcal{V}$. $\rho_V^d(t)$ designates the state of V at time $t \in [0, d)$. For brevity, we write $\text{Left}(\rho_V^d) \triangleq \rho_V^d(0)$ and $\text{Right}^-(\rho_V^d) \triangleq \lim_{t \rightarrow d^-} \rho_V^d(t)$ for the left and right-limit states of ρ_V^d , respectively. The name set and the duration of ρ_V^d are obtained by $\mathcal{N}(\rho_V^d) = V$ and $|\rho_V^d| = d$. Therefore, in what follows, we omit V and d and use ρ to denote a flow. We require ρ to be *right continuous* and *semi-differentiable* within its time domain. Additionally, the left limit at the end point of ρ should exist.

Two flows ρ and ρ' can be concatenated, denoted $\rho \frown \rho'$, if the right limit of ρ and the initial state of ρ' coincide, i.e. $\text{Right}^-(\rho) = \text{Left}(\rho')$. Formally, $(\rho \frown \rho')(t) = \rho(t)$ if $0 \leq t < |\rho|$ and $\rho'(t - |\rho|)$ if $|\rho| \leq t < |\rho| + |\rho'|$. The union and intersection of two flows are defined if they have the same duration and agree on the common names. We define $\rho \uplus \rho' = \rho \cup \rho'$ if $\mathcal{N}(\rho) \cap \mathcal{N}(\rho') = \emptyset$. In addition, $\rho - \rho' = \rho''$ if $\rho = \rho'' \uplus (\rho \cap \rho')$.

A flow ρ can be refined as a pair $\mathcal{A} \cdot \mathcal{G}$, logically equivalent to $\mathcal{A} \uplus \mathcal{G}$, where $\mathcal{G}$ is the flow of the continuous variables defined in the process under consideration and $\mathcal{A}$ is the flow of the (undefined) continuous variables it uses. In other words, $\mathcal{A}$ is the trajectory of continuous variables defined in the environment with which the process interacts. In this sense, $\mathcal{A}$ and $\mathcal{G}$ may be called the *assumption* and *guarantee*, respectively, in terms borrowed from contract theory [8]. A flow $\mathcal{A} \cdot \mathcal{G}$ is said to be *closed* if $\mathcal{N}(\mathcal{A}) = \emptyset$, i.e., it has no assumption on the environment. In control theory, it is hence called an autonomous system (the actual implementation of a physical process). The composition of flows can algebraically be defined as the composition of contracts:

$$(\mathcal{A} \cdot \mathcal{G}) \otimes (\mathcal{A}' \cdot \mathcal{G}') \triangleq ((\mathcal{A} - \mathcal{G}') \cup (\mathcal{A}' - \mathcal{G})) \cdot (\mathcal{G} \cup \mathcal{G}')$$

In the remainder, we use ρ and $\mathcal{A} \cdot \mathcal{G}$ to denote a flow, interchangeably. We say a set of ODEs $\dot{\vec{v}} = \vec{e}$ evolves along a flow $\mathcal{A} \cdot \mathcal{G}$, denoted $\llbracket \dot{\vec{v}} = \vec{e} \rrbracket_{\mathcal{A} \cdot \mathcal{G}}$, iff (1) $\mathcal{N}(\mathcal{G}) = \mathcal{N}(\vec{v})$ and $\mathcal{N}(\mathcal{A}) = \mathcal{N}(\vec{e}) - \mathcal{N}(\vec{v})$,

and (2) $\forall t \in (0, |\mathcal{A} \cdot \mathcal{G}|) \cdot \dot{\mathcal{G}}(t)(\vec{v}) = \llbracket \vec{e} \rrbracket_{(\mathcal{A} \cdot \mathcal{G})(t)}$. Similarly, we say a Boolean expression B keeps holding along a flow ρ , denoted $\llbracket B \rrbracket_\rho$, iff $\forall t \in [0, |\rho|) \cdot \llbracket B \rrbracket_{\rho(t)} = \text{true}$.

5.4 A Structural Operational Semantics

The operational semantics is defined by a proof system that consists of two rule-sets:

- (1) A set of discrete transition rules, for example, $P \xrightarrow{\tau} Q$.
- (2) A set of, so called, continuous evolution rules, for example, $P \xrightarrow{d} Q$ where $d > 0$.

While the first one defines transitions that can conceptually be perceived as taking “discrete” or no time to perform an action, like a conventional labelled transition system, the second rule-set does, on the contrary, define the continuous trajectories of an hybrid system over time.

5.4.1 Abstractions and Concretions. To express the discrete transition relation correctly, we introduce the notions of *abstractions* and *concretions* due to [35].

Definition 5.1 (Abstraction and concretion). $(\vec{x}).P$ is an *abstraction* of process P . Two abstractions are structurally congruent ($\equiv$) if, up to α -conversion, their bound names $\vec{x}$ are identical and their processes are structurally congruent. Conversely, $(\nu \vec{y})\langle \vec{e} \rangle.P$ is a *concretion* of process P with $\mathcal{N}(\vec{y}) \subseteq \mathcal{N}(\vec{e})$. Two concretions are structurally congruent if, up to alpha-conversion and re-ordering of restricted names, their prefixes $(\nu \vec{y})\langle \vec{e} \rangle$ are identical and their processes are structurally congruent.

We highlight *abstractions* as F or G and *concretions* as C or D , as they have a specific usage. An agent A or B is either an abstraction or a concretion. An ordinary process P is a special kind of agent: it is both an abstraction $()P$ and a concretion $\langle \rangle P$.

Definition 5.2 (Ions). Abstraction and a concretion can be regarded as positive and negative *ions*. Assuming $y \notin \mathcal{N}(\vec{x})$ and $\mathcal{N}(\vec{x}) \cap \mathcal{F}(Q) = \emptyset$, then:

$$\begin{aligned} (\nu y)(\vec{x}).P &\triangleq (\vec{x}).(\nu y)P \text{ and } ((\vec{x}).P) \parallel Q \triangleq (\vec{x}).(P \parallel Q) \\ ((\nu \vec{x})\langle \vec{e} \rangle.P) \parallel Q &\triangleq (\nu \vec{x})\langle \vec{e} \rangle.(P \parallel Q) \text{ and } (\nu y)((\nu \vec{x})\langle \vec{e} \rangle.P) \triangleq \begin{cases} (\nu y, \vec{x})\langle \vec{e} \rangle.P & \text{if } y \in \mathcal{N}(\vec{e}) \\ (\nu \vec{x})\langle \vec{e} \rangle.(\nu y)P & \text{otherwise} \end{cases} \end{aligned}$$

Definition 5.3 (Application). The application of an abstraction and a concretion is defined by $(\vec{x}).P @ (\nu \vec{y})\langle \vec{e} \rangle.Q \triangleq (\nu \vec{y})(P\{\vec{e}/\vec{x}\} \parallel Q)$, assuming that $\mathcal{N}(\vec{y}) \cap \mathcal{F}(P) \subseteq \mathcal{N}(\vec{x})$. In particular, for an abstraction $F = (\vec{x}).P$, we write $F\langle \vec{e} \rangle$ for $F @ \langle \vec{e} \rangle$ to mean $P\{\vec{e}/\vec{x}\}$.

5.4.2 Small-step Operational Semantics. The transition of a process P to an agent A performing the action α is defined by $P \xrightarrow{\alpha} A$. The action α can be discrete or continuous:

- (1) A discrete action λ , which can either be silent (τ), an input x , or an output $\bar{x}$;
- (2) A continuous action $\langle \rho, \mathcal{R} \rangle$, which describes the continuous trajectory defined by a flow ρ and a ready set of channels $\mathcal{R}$, pending communication during the evolution.

The operational semantics is defined by the rule-sets of Table 1, for discrete transitions, and of Table 2, for continuous evolution. We first define the structural equivalence rule. It stipulates that, if two processes are structurally congruent, then they have the same behaviour. Rule [EQV] provides alpha-equivalence, commutativity, associativity to both transition systems: it is labelled by α .

$$\frac{P \equiv Q \quad Q \xrightarrow{\alpha} B \quad B \equiv A}{P \xrightarrow{\alpha} A} [\text{EQV}]$$

Rule $[\lambda\text{-Act}]$. Top-left of Table 1, a sum is represented as $\lambda A + M$, where λ is a discrete action (silence τ , input x , or output $\bar{x}$) and A is an agent (abstraction, concretion, or process). The agent A is triggered if λ is executed.

Rule [PASS]. If the Boolean expression B evaluates to **true**, the process $[B].P + M$ silently transitions to P . Guards allow to define **if** B **then** P **else** Q by $[B].P + [\neg B].Q$

Rule [SENSE]. A process of continuous prefix can be selected for sensing to allow an ODE be interrupted and sensed by the environment. Assume that v is the i -th element of $\vec{v}$ in the ODE: $v = \vec{v}(i)$. Then, the output action $\bar{v}$ in the interface $\mathcal{R}$ indicates that the current value of v : $e = \vec{e}_0(i)$ can be sent out by $\bar{v}$. Thus, the continuous process transitions to a concretion whose “charge” is $\langle e \rangle$ by the output action $\bar{v} \in \mathcal{R}$. Notice that sensing does not stop the ODE. After merging the “charge” $\langle e \rangle$ with the environment, the ODE continues to evolve from its current state $\vec{e}_0$.

Rule [ACTUATE]. Actuation is symmetric to sensing: a process of continuous prefix can be interrupted for actuation. The presence of an input action v in the interface $\mathcal{R}$ indicates that the current value of v , i.e., $\vec{e}_0(i)$, can be modified to a new value, denoted by a fresh variable x . When that happens, the i -th element of $\vec{e}_0$ is replaced with x , represented by $\vec{e}_1 = \vec{e}_0[i \mapsto x]$ (to mean that $\vec{e}_1(i) = x$ and $\vec{e}_1(j) = \vec{e}_0(j)$ for all $j \neq i$). In the conclusion of the rule, the continuous process transitions to an abstraction whose “charge” is (x) by an input action $v \in \mathcal{R}$. Similarly, actuation does not stop an ODE, i.e., once an actual value substitutes the “charge” (x) by composition, the resumes its evolution with its updated state.

Table 1. Discrete transitions.

$$\begin{array}{c}
 \frac{}{\lambda A + M \xrightarrow{\lambda} A} [\lambda\text{-ACT}] \quad \frac{\llbracket B \rrbracket = \mathbf{true}}{[B].P + M \xrightarrow{\tau} P} [\text{PASS}] \\
 \\
 \frac{\bar{v} \in \mathcal{R} \quad v = \vec{v}(i) \quad \vec{e}_0(i) = e}{\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P + M \xrightarrow{\bar{v}} \langle e \rangle. \{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P} [\text{SENSE}] \\
 \\
 \frac{v \in \mathcal{R} \quad v = \vec{v}(i) \quad \vec{e}_1 = \vec{e}_0[i \mapsto x] \quad x \notin \mathcal{N}(\vec{e}_0, \vec{v}, \vec{e}, B, \vec{z}) \cup \mathcal{N}(P)}{\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P + M \xrightarrow{v} (x). \{\vec{e}_1 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P} [\text{ACTUATE}] \\
 \\
 \frac{P \xrightarrow{\lambda} A}{P \parallel Q \xrightarrow{\lambda} A \parallel Q} [\text{PAR}] \quad \frac{P \xrightarrow{x} F \quad Q \xrightarrow{\bar{x}} C}{P \parallel Q \xrightarrow{\tau} F @ C} [\text{SYNC}] \quad \frac{P \xrightarrow{\lambda} A \quad \lambda \notin \{x, \bar{x}\}}{(\nu x)P \xrightarrow{\lambda} (\nu x)A} [\text{RES}] \quad \frac{P \parallel !P \xrightarrow{\lambda} A}{!P \xrightarrow{\lambda} A} [\text{REP}]
 \end{array}$$

Rule [SYNC]. As for the rule of sensing and actuation, an input action x produces an abstraction F which synchronises with the concretion C of the matching output action $\bar{x}$. It results in a silent action τ and the application $F @ C$.

Rule [PAR]. Parallel processes $P \parallel Q$ may also evolve independently w.r.t. a discrete action λ .

Rule [RES]. Restriction $(\nu x)P$ behaves as P if its discrete action λ does not concern the name x , i.e., $\lambda \notin \{x, \bar{x}\}$. The rule stipulates that references to x cannot escape the scope of $(\nu x)(\cdot)$.

Rule [REP]. The meaning of replication $!P$ reflects the intuition of forking an unlimited number of copies of P in parallel. Replication can model recursion as $\mu x(\vec{y}).P @ \langle \vec{e} \rangle \triangleq (\nu x)(\bar{x} \langle \vec{e} \rangle \parallel !x(\vec{y}).P)$. For convenience, we write $\mu x.P$ for $\mu x().P @ \langle \rangle$ as, e.g., in the recursion **Ground** in Section 3.1.

The rule-sets in Table 2 define the continuous behaviours of all (hybrid) processes.

Rule [RUN]. A run describes the evolution of a continuous prefix $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e} \& B, \mathcal{R}\}$. It starts from the initial state represented by a vector of expressions $\vec{e}_0$ and then evolves according to the (set of) ODE(s) $\dot{\vec{v}} = \vec{e}$. Thus, $\vec{e}_0$ should be evaluated with respect to the initial state of $\mathcal{S}$, i.e., $\text{Left}(\mathcal{S})(\vec{v}) = \llbracket \vec{e}_0 \rrbracket$. Then, the derivative $\dot{\vec{v}} = \vec{e}$ must evaluate to **true** along the entire trajectory $\mathcal{A} \cdot \mathcal{S}$, i.e., $\llbracket \dot{\vec{v}} = \vec{e} \rrbracket_{\mathcal{A} \cdot \mathcal{S}}$. The state of the ODE evolves within the boundary defined by B , i.e., $\llbracket B \rrbracket_{\mathcal{A} \cdot \mathcal{S}}$.

while the channels in the ready set $\mathcal{R}$ listen to the environment. The final state of $\vec{v}$ of the evolution is defined by the limit $\text{Right}^-(\mathcal{G})(\vec{v}) = \vec{c}$. However, the premise $\llbracket B \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}$ should still hold when that happens, allowing the process to resume from state $\vec{c}$ and, e.g., an interruption using rules [SENSE] or [ACTUATE] of Table 1.

Rule [STOP]. Continuous evolution stops once it reaches the boundary of B . Starting from the initial condition $\llbracket \vec{e}_0 \rrbracket$, the ODE evolves along the trajectory $\mathcal{A} \cdot \mathcal{G}$ until it reaches its boundary, i.e., $\neg \llbracket B \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}$, unlike rule [RUN]. Once the evolution stops, the current state $\vec{c} = \text{Right}^-(\mathcal{G})(\vec{v})$ of the ODE is transferred to its continuation, $(\vec{y}).P$, i.e., $(\vec{y}).P @ \langle \vec{c} \rangle \equiv P\{\vec{c}/\vec{y}\}$.

Table 2. Continuous evolution.

$$\begin{array}{c}
\frac{\text{Left}(\mathcal{G})(\vec{v}) = \llbracket \vec{e}_0 \rrbracket \quad \text{Right}^-(\mathcal{G})(\vec{v}) = \vec{c} \quad \llbracket \dot{v} = \vec{e} \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket B \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket B \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}}{\{\vec{e}_0 \mid \dot{v} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P + M \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \mathcal{R} \rangle} \{\vec{c} \mid \dot{v} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P} [\text{RUN}] \\
\\
\frac{\text{Left}(\mathcal{G})(\vec{v}) = \llbracket \vec{e}_0 \rrbracket \quad \text{Right}^-(\mathcal{G})(\vec{v}) = \vec{c} \quad \llbracket \dot{v} = \vec{e} \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket B \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \neg \llbracket B \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}}{\{\vec{e}_0 \mid \dot{v} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P + M \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \mathcal{R} \rangle} P\{\vec{c}/\vec{y}\}} [\text{STOP}] \\
\\
\frac{\forall i \in I. \lambda_i \neq \tau}{\sum_{i \in I} \lambda_i A_i \xrightarrow{\langle \rho_0^d, \{\lambda_i\}_{i \in I} \rangle} \sum_{i \in I} \lambda_i A_i} [\text{WAIT}] \quad \frac{P \xrightarrow{\langle \rho, \mathcal{R} \rangle} P' \quad Q \xrightarrow{\langle \rho', \mathcal{R}' \rangle} Q' \quad \overline{\mathcal{R}} \cap \mathcal{R}' = \emptyset}{P \parallel Q \xrightarrow{\langle \rho \otimes \rho', \mathcal{R} \cup \mathcal{R}' \rangle} P' \parallel Q'} [\text{PAR}'] \\
\\
\frac{P \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \mathcal{R} \rangle} P' \quad x \notin \mathcal{N}(\mathcal{A})}{(\nu x)P \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \mathcal{R} \rangle \setminus x} (\nu x)P'} [\text{RES}'] \quad \frac{P \xrightarrow{\langle \rho, \mathcal{R} \rangle} P' \quad \overline{\mathcal{R}} \cap \mathcal{R} = \emptyset}{!P \xrightarrow{\langle \rho, \mathcal{R} \rangle} !P'} [\text{REP}']
\end{array}$$

Example 5.4. Consider $\{1 \mid \dot{v} = v \& v < 5\}(y).P$, where the solution of $\{\dot{v} = v\}$ with initial condition $v(0) = 1$ is $v(t) = \exp(t)$ for $t > 0$. It may evolve for $\ln(5)$ time units and then stop to execute the continuation by rule [STOP]:

$$\frac{\text{Left}(\mathcal{G})(v) = \llbracket 1 \rrbracket \quad \text{Right}^-(\mathcal{G})(v) = 5 \quad \llbracket \dot{v} = v \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket v < 5 \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \neg \llbracket v < 5 \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}}{\{1 \mid \dot{v} = v \& v < 5\}(y).P \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \emptyset \rangle} P\{5/y\}}$$

where $\mathcal{A} \cdot \mathcal{G}$ is the flow with $|\mathcal{A} \cdot \mathcal{G}| = \ln(5)$, $\mathcal{N}(\mathcal{A}) = \emptyset$, $\mathcal{N}(\mathcal{G}) = \{v\}$, and $\mathcal{G}(t)(v) = \exp(t)$. Also, the process may evolve for mere 1 time units by rule [RUN], i.e., $|\mathcal{A} \cdot \mathcal{G}| = 1$, $\mathcal{N}(\mathcal{A}) = \emptyset$, $\mathcal{N}(\mathcal{G}) = \{v\}$, and $\mathcal{G}(t)(v) = \exp(t)$. The target process may then evolve for $\ln(5) - 1$ time units to reach $P\{5/y\}$, i.e., the flow from $\{1 \mid \dot{v} = v \& v < 5\}(y).P$ to $P\{5/y\}$ is chopped into two segments.

$$\frac{\text{Left}(\mathcal{G})(v) = \llbracket 1 \rrbracket \quad \text{Right}^-(\mathcal{G})(v) = \exp(1) \quad \llbracket \dot{v} = v \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket v < 5 \rrbracket_{\mathcal{A} \cdot \mathcal{G}} \quad \llbracket v < 5 \rrbracket_{\text{Right}^-(\mathcal{A} \cdot \mathcal{G})}}{\{1 \mid \dot{v} = v \& v < 5\}(y).P \xrightarrow{\langle \mathcal{A} \cdot \mathcal{G}, \emptyset \rangle} \{\exp(1) \mid \dot{v} = v \& v < 5\}(y).P}$$

Rule [WAIT]. In the continuous semantics, a sum $\sum_{i \in I} \lambda_i A_i$ holding discrete input and output prefixes (i.e. $\lambda_i \neq \tau$) is regarded as choice waiting for an interruption to happen. To that effect, its prefixes are added to the ready set as $\{\lambda_i\}_{i \in I}$ and its flow is empty (ρ_0^d).

Rule [PAR]. Two processes evolving in parallel, $P \parallel Q$, can synchronise over the same period of time by jointly orchestrating the “contractual” composition $\rho \otimes \rho'$, if one is not waiting for the other to communicate, i.e., $\overline{\mathcal{R}} \cap \mathcal{R}' = \emptyset$, where $\overline{\mathcal{R}} \triangleq \{\overline{x}, y \mid x, \overline{y} \in \mathcal{R}\}$ is the dual of $\mathcal{R}$. Conversely, if $\overline{\mathcal{R}} \cap \mathcal{R}' \neq \emptyset$, both parties are waiting for a discrete communication to occur with a matching

channel. This must instead be done immediately, by using rules [SENSE], [ACTUATE] or [SYNC], instead of rule [PAR'], which does not apply in that case.

Example 5.5. Consider the parallel composition of $x(y).P$ and $\bar{x}\langle 1 \rangle.Q$, it can be resolved by using rule [SYNC] in Table 1, instead of [PAR'] in Table 2, because the ready sets match: $\overline{\{x\}} \cap \{\bar{x}\} = \{\bar{x}\}$.

Rule [RES']. Similarly, if P can perform the trajectory $\mathcal{A} \cdot \mathcal{G}$ if the scope of x is restricted to P (it is not an assumption $\mathcal{A}$), then $(\nu x)P$ can perform the same trajectory while keeping the name x private to P , represented by $\langle \mathcal{A} \cdot \mathcal{G}, \mathcal{R} \rangle \setminus x$, which is equivalent to $\langle \mathcal{A} \cdot \mathcal{G} \setminus x, \mathcal{R} - \{x, \bar{x}\} \rangle$, where $\mathcal{G} \setminus x = \mathcal{G}$ if $x \notin N(\mathcal{G})$ and $\mathcal{G}'$ if $\mathcal{G} = \mathcal{G}' \uplus \mathcal{G}_x$ with $N(\mathcal{G}_x) = \{x\}$.

Rule [REP']. A continuous transition from P to P' implies one from $!P$ to $!P'$. Intuitively, it is a special case of rule [PAR'], since $!P$ can actually be seen as copies of P executing in parallel.

REMARK 3. A continuous action of the form $\langle \rho_0^d, \emptyset \rangle$ denotes an evolution that lasts for d time, whose trajectory cannot be observed (it has no observable variable in lexical scope) and whose ready set is empty. It hence can be treated by its side-effect: a delay of time, and simplified as d , the duration of the delay. For example, the transition from `wait`(d) in Section 3 may adopt this simplification.

6 Bisimulation and Congruence

The **HpC** is a conservative extension of the classical π -calculus. As we have seen, the extension is operationally “minimal”, and is yet able to describe mobility and physics while allowing to lift existing theoretical results (e.g. bisimulation) of the classic π -calculus [35] to the context of that extension. We first define strong and weak bisimulations in the **HpC**, extended to hybrid systems, and prove some of its desired properties. Then, we define the notion of approximate bisimulation, which can be treated as the generalised form of weak bisimulation. It is worth mentioning that approximate bisimulation is, to date, only known for first-order calculi (i.e. without process dynamicity or channel mobility).

6.1 Strong Bisimulation and Congruence

Definition 6.1 (Strong Bisimulation). Let F and G denote abstractions and C and D concretions, a binary relation S over the processes is a strong simulation if, for each PSQ we have

- $P \xrightarrow{\tau} P'$ implies $\exists Q' \text{ s.t. } Q \xrightarrow{\tau} Q' \text{ and } P'SQ'$;
- $P \xrightarrow{x} F$ implies $\exists G \text{ s.t. } Q \xrightarrow{x} G \text{ and } FSG$;
- $P \xrightarrow{\bar{x}} C$ implies $\exists D \text{ s.t. } Q \xrightarrow{\bar{x}} D \text{ and } CSD$;
- $P \xrightarrow{\langle \rho_1 \frown \dots \frown \rho_n, \mathcal{R} \rangle} P'$ implies $\exists Q' \text{ s.t. } Q \xrightarrow{\langle \rho_1, \mathcal{R} \rangle} \dots \xrightarrow{\langle \rho_n, \mathcal{R} \rangle} Q' \text{ and } P'SQ'$.

The S -relations between abstractions and concretions are defined below:

- FSG means that $F\langle \vec{e} \rangle SG\langle \vec{e} \rangle$ for all $\vec{e}$;
- CSD means that $C \equiv (\nu \vec{y})\langle \vec{e} \rangle.P$ and $D \equiv (\nu \vec{y})\langle \vec{e} \rangle.Q$ s.t. PSQ .

If both S and its inverse are strong simulations then S is a strong bisimulation. Two agents A and B are strongly equivalent, written $A \sim B$, if (A, B) belongs to a strong bisimulation.

The following two theorems prove that $\sim$ for the **HpC** is (1) an equivalence relation, i.e., $P \sim P$ (reflexivity), $P \sim Q$ implies $Q \sim P$ (symmetry), and $P \sim Q$ and $Q \sim R$ imply $P \sim R$ (transitivity); (2) a strong bisimulation, i.e., relation $S = \{(P, Q) \mid P \sim Q\}$ is a strong bisimulation, where P and Q are **HpC** processes; (3) a congruence, i.e., $P \sim Q$ implies $\mathbb{C}[P] \sim \mathbb{C}[Q]$ for any context $\mathbb{C}$.

THEOREM 6.2 (EQUIVALENCE AND STRONG BISIMULATION).

(1) $\sim$ is an equivalence relation, and, (2) $\sim$ is itself a strong bisimulation.

PROOF. (1) For reflexivity, it is enough to show that the identity relation over the set of agents $\mathcal{A}$, i.e., the relation $\{(A, A) \mid A \in \mathcal{A}\}$ is a strong bisimulation. For symmetry, we have to show that if S is a strong bisimulation then so is its converse S^{-1} , and this is obvious from the definition. For transitivity, we must show that if S and S' are strong bisimulations, then so is their composition

$$SS' \triangleq \{(A, B) \mid \exists X. ASX \wedge XS'B\}$$

It is enough to show that this is a strong simulation. Let $(A, B) \in SS'$. Then,

Case 1 A and B are abstractions. According to Definition 6.1, $A\langle\vec{e}\rangle SX\langle\vec{e}\rangle$ and $X\langle\vec{e}\rangle S'B\langle\vec{e}\rangle$ for all $\vec{e}$, implying $A\langle\vec{e}\rangle SS'B\langle\vec{e}\rangle$ for all $\vec{e}$.

Case 2 A and B are concretions. Let $A \equiv (v\vec{y})\langle\vec{e}\rangle.P$, $B \equiv (v\vec{y})\langle\vec{e}\rangle.Q$, and $X \equiv (v\vec{y})\langle\vec{e}\rangle.R$. Then, according to Definition 6.1, PSR and $RS'Q$, implying $PS'S'Q$.

Case 3 A and B are processes. Let $A \xrightarrow{\alpha} A'$. Then,

- $\alpha = \lambda$ is a discrete action. By Definition 6.1, $\exists X'$ such that $X \xrightarrow{\lambda} X'$ and $A'SX'$, and hence $\exists B'$ such that $B \xrightarrow{\lambda} B'$ and $X'S'B'$. Thus, $A \xrightarrow{\lambda} A'$ implies $B \xrightarrow{\lambda} B'$ and $A'SS'B'$.
- $\alpha = \langle\rho, \mathcal{R}\rangle$ is a continuous action. By Definition 6.1, $\exists X'$ s.t. $X \xrightarrow{\langle\rho_1, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n, \mathcal{R}\rangle} X'$ and $A'SX'$, where $\rho = \rho_1 \frown \dots \frown \rho_n$, and hence $\exists B'$ s.t.

$$B \xrightarrow{\langle\rho_1^{(1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_1^{(m_1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n^{(1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n^{(mn)}, \mathcal{R}\rangle} B'$$

and $X'S'B'$, where $\rho_i = \rho_i^{(1)} \frown \dots \frown \rho_i^{(m_i)}$ for $1 \leq i \leq n$. Thus,

$$A \xrightarrow{\langle\rho_1^{(1)} \frown \dots \frown \rho_1^{(m_1)} \frown \dots \frown \rho_n^{(1)} \frown \dots \frown \rho_n^{(mn)}, \mathcal{R}\rangle} A'$$

implies $B \xrightarrow{\langle\rho_1^{(1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_1^{(m_1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n^{(1)}, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n^{(mn)}, \mathcal{R}\rangle} B'$ and $A'SS'B'$.

In summary, the strong simulation condition for SS' is established.

(2) Let $A \sim B$, where A and B are agents. Then by definition ASB for some strong bisimulation S .

Case 1 If $A \xrightarrow{\lambda} A'$, then $\exists B'$ such that $B \xrightarrow{\lambda} B'$ and $A'SB'$;

Case 2 If $A \xrightarrow{\langle\rho_1, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n, \mathcal{R}\rangle} A'$, then $\exists B'$ such that $B \xrightarrow{\langle\rho_1, \mathcal{R}\rangle} \dots \xrightarrow{\langle\rho_n, \mathcal{R}\rangle} B'$ and $A'SB'$.

Hence also $A' \sim B'$. Thus, $\sim$ is a strong simulation, and by symmetry so is its converse. $\square$

THEOREM 6.3 (CONGRUENCE).

- If $A \sim B$ then $\lambda A + M \sim \lambda B + M$.
- If $P \sim Q$ then (1) $[B].P + M \sim [B].Q + M$, (2) $(vx)P \sim (vx)Q$, (3) $P \parallel R \sim Q \parallel R$, (4) $!P \sim !Q$, (5) $(v\vec{x})\langle\vec{e}\rangle.P \sim (v\vec{x})\langle\vec{e}\rangle.Q$, and (6) $\{\vec{e}_0 \mid \vec{v} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).P + M \sim \{\vec{e}_0 \mid \vec{v} = \vec{e} \& B, \mathcal{R}\}(\vec{y}).Q + M$.
- If $P\{\vec{e}/\vec{x}\} \sim Q\{\vec{e}/\vec{x}\}$ for all $\vec{e}$, then $(\vec{x}).P \sim (\vec{x}).Q$.

PROOF. The detailed proof can be found in [52, Appendix A] $\square$

PROPERTY 6.1 (STRONG EQUIVALENCE).

- (1) $(vx)\emptyset \sim \emptyset$, $P \parallel \emptyset \sim P$, and $M + \emptyset \sim M$.
- (2) $(vx)(P \parallel Q) \sim P \parallel (vx)Q$ if $x \notin \mathcal{F}(P)$.
- (3) $(vx)(vy)P \sim (vy)(vx)P$.

PROOF. The property is proved by case analysis on the semantics rules of Tables 1 and 2 using Definition 6.1. $\square$

The above equivalence relations usually appear as the structural congruence rules of π -calculus. Here we can prove that they are strong equivalences and hence removed from the structural congruence rules for the **HpC** (Section 4.4).

6.2 Weak Bisimulation

Intuitively, the difference between strong and weak bisimulation is that the latter disregards τ -actions, as τ -actions usually denote internal or invisible behaviours such as internal communications and discrete computations whose time can be ignored.

We use $\Rightarrow$ to denote finite τ -actions. Concretely, $P \Rightarrow P'$ if $P \xrightarrow{\tau} \dots \xrightarrow{\tau} P'$. Besides, $P \xrightarrow{x\vec{e}} P'$ is defined if $P \xrightarrow{x} F$ and $F(\vec{e}) = P'$. Then, we can define the weak bisimulation as follows:

Definition 6.4 (Weak Bisimulation). A binary relation S over the processes is a weak simulation if, for each PSQ we have

- $P \xrightarrow{\tau} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow Q'$ and $P'SQ'$;
- $P \xrightarrow{x\vec{e}} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{x\vec{e}} Q'$ and $P'SQ'$;
- $P \xrightarrow{\bar{x}} (v\vec{y})\langle\vec{e}\rangle.P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{\bar{x}} (v\vec{y})\langle\vec{e}\rangle.Q''$, $Q'' \Rightarrow Q'$, and $P'SQ'$;
- $P \xrightarrow{\langle\rho_1 \dots \rho_n, R\rangle} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{\langle\rho_1, R\rangle} \dots \Rightarrow \xrightarrow{\langle\rho_n, R\rangle} Q'$ and $P'SQ'$.

If both S and its inverse are weak simulations then S is a weak bisimulation. Two processes P and Q are weakly equivalent, written $P \approx Q$, if (P, Q) is in some weak bisimulation.

The last rule of Definition 6.4 extends classical weak bisimulation to continuous actions that are “chopped” by discrete τ -actions. For example, $\text{wait}(3) \approx \text{wait}(1).\tau.\text{wait}(1).\tau.\tau.\text{wait}(1)$. The following theorem reveals that $\approx$ is an equivalence relation and itself a weak bisimulation.

THEOREM 6.5 (EQUIVALENCE AND WEAK BISIMULATION).

- (1) $\approx$ is an equivalence relation;
- (2) $\approx$ is itself a weak bisimulation.

PROOF. It is analogous to Theorem 6.2. □

6.3 Approximate Bisimulation

The notion of approximate bisimulation was first proposed in [19] to suit the study of hybrid systems. It allows the observations of two systems to be distant within a tolerable bound, rather than exactly identical. In [24], the approximate bisimulation was extended to allow precision not only between the observations of two systems, but also between the timely synchronisation between them. The work of [54] extends approximate bisimulation by allowing both a tolerance on time and value between the observations of two systems. It can be used to prove the approximate equivalence between a hybrid discrete-continuous model (in, e.g., Hybrid CSP) and the discrete SystemC [54] and C [49] code generated from it.

Definition 6.6 (Approximate Bisimulation). A binary relation S over the processes is an (ϵ, δ) -approximate simulation if for each PSQ we have:

- $P \xrightarrow{\tau} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow Q'$ and $P'SQ'$;
- $P \xrightarrow{x\vec{e}} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{x\vec{e}} Q'$ and $P'SQ'$;
- $P \xrightarrow{\bar{x}} (v\vec{y})\langle\vec{e}\rangle.P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{\bar{x}} (v\vec{y})\langle\vec{e}\rangle.Q''$, $Q'' \Rightarrow Q'$, and $P'SQ'$;
- $P \xrightarrow{\langle\rho, R\rangle} P'$ implies $\exists Q'$ s.t. $Q \Rightarrow \xrightarrow{\langle\rho_1, R_1\rangle} \dots \Rightarrow \xrightarrow{\langle\rho_n, R_n\rangle} Q'$, and $P'SQ'$, where $\|\text{Left}(\mathcal{G}) - \text{Left}(\mathcal{G}_1)\| \leq \epsilon$ and $\|\rho\| - \|\rho_1 \frown \dots \frown \rho_n\| \leq \delta$ with $\rho = \mathcal{A} \cdot \mathcal{G}$ and $\rho_1 = \mathcal{A}_1 \cdot \mathcal{G}_1$.

If both $\mathcal{S}$ and its converse are (ϵ, δ) -approximate simulations then $\mathcal{S}$ is an (ϵ, δ) -approximate bisimulation. $\mathcal{P}$ and $\mathcal{Q}$ are (ϵ, δ) -approximately bisimilar, written $\mathcal{P} \approx_{\epsilon, \delta} \mathcal{Q}$, if $(\mathcal{P}, \mathcal{Q})$ is in some (ϵ, δ) -approximate bisimulation.

In particular, the last rule says that, if two processes $\mathcal{P}$ and $\mathcal{Q}$ are in (ϵ, δ) -approximate bisimulation, then (1) the distance between observations of $\mathcal{P}$ and $\mathcal{Q}$ should not exceed ϵ (the left states of guaranteed flows $\mathcal{G}$ and $\mathcal{G}_1$ of continuous variables defined by $\mathcal{P}$ and $\mathcal{Q}$ denote their observed states, respectively). Then, (2) the distance between two continuous actions of $\mathcal{P}$ and $\mathcal{Q}$ (their difference in length) should not exceed δ . Finally, (3) the target processes $\mathcal{P}'$ and $\mathcal{Q}'$ should be in the same bisimulation. Notice that weak bisimulation in Definition 6.4 is a $(0, 0)$ -approximate bisimulation. The distance allowed between states in approximate bisimulation can be considered as a kind of metrics in quantitative bisimulation [18, 27], which is however limited to states and traces. In both frameworks, its goal is to define a safe limit for the update period of the controller.

The discretisation of a hybrid model plays an important role in the model-based design of cyber-physical systems. To design a system combined with discrete, continuous and mobility, code at the implementation-level must determine how to sample data from the continuous plant, compute control commands, and perform the interactions between these different parts correctly. An efficient solution is to first have a hybrid control model that can be formally verified with safety guarantee, and then to discretise the continuous plant to obtain an algorithmic model to implement the cyber-physical system.

The notion of approximate bisimulation provides a bridge between hybrid models in control theory and discrete code in computer science. When a weak bisimulation relation between the hybrid model and its discretisation holds, the correctness of the code implementation is guaranteed. Actually, based on the approximate bisimulation of Definition 6.6, the discretisation of the whole **HpC** to correct executable code can be done similarly as for HCSP in [49, 54]. For example, consider the discretisation of the continuous evolution of time length $d > 0$. Concretely, the continuous process $\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e}\}_d \triangleq (\nu \mathbf{c})\{0, \vec{e}_0 \mid \dot{\mathbf{c}} = 1, \dot{\vec{v}} = \vec{e} \ \& \ \mathbf{c} < d\}$ evolves from $\vec{e}_0$ according to $\dot{\vec{v}} = \vec{e}$ for d time units. Its discretisation is given by the following theorem.

THEOREM 6.7 (DISCRETISATION). *Given a set of ODEs $\dot{\vec{v}} = \vec{e}$, with $\mathcal{N}(\vec{e}) \subseteq \mathcal{N}(\vec{v})$ and $\vec{e}_0$ as the initial value, satisfying the local Lipschitz condition on the interval $[0, d]$ for some $d > 0$, i.e., there exists a constant $L > 0$ such that $\|\vec{e}\{\vec{c}_1/\vec{v}\} - \vec{e}\{\vec{c}_2/\vec{v}\}\| \leq L\|\vec{c}_1 - \vec{c}_2\|$ for all $\vec{c}_1, \vec{c}_2 \in \mathbb{D}$, where $\mathbb{D} \subset \mathbb{R}^{|\vec{v}|}$ is any compact set. Let $\vec{c}$ be the values satisfying $\|\vec{c} - \vec{e}_0\| \leq \epsilon'$, then for any precision $\epsilon > \epsilon' > 0$, there exists a step size $\delta > 0$, using coefficients Φ of the 4-stage Runge-Kutta method for computing approximated values of $\vec{v}$, such that $\text{step}(\vec{y}, d) \triangleq (\nu \mathbf{c})\{0, \vec{y} \mid \dot{\mathbf{c}} = 1, \dot{\vec{v}} = \vec{0} \ \& \ \mathbf{c} < d\}$ and*

$$\{\vec{e}_0 \mid \dot{\vec{v}} = \vec{e}\}_d \approx_{(\epsilon, \delta)} \mu x(\vec{y}, z). \left([z \geq \delta]. \text{step}(\vec{y}, \delta). \bar{x}(\vec{y} + \Phi(\vec{y}, \delta) \cdot \delta, z - \delta) \right. \\ \left. + [0 < z < \delta]. \text{step}(\vec{y}, z) + [z = 0]. 0 \right) @ \langle \vec{c}, d \rangle$$

7 Case Study: A Network's Handover Protocol

To put all the features and definitions of the **HpC** in motion, our case study is that of a handover protocol for next-generation's high-speed train control system. For the sake of clarity, we abstract away most of the design details and actual system parameters to only highlight key behaviours of the protocol, assuming wired or wireless private communications (i.e. not open broadcast radio channels, for obvious safety and security concerns).

7.1 Overview

The overall framework of the train control system is shown in Fig. 1. The network is composed of a train, of several railway-sectors, and of a terminus where the train stops. The train starts from the departure and ends at the terminus, under the coordinated control of the consecutive sectors.

The railway sector can directly control the train by sensing and actuating using private channels of the train (its ready-set interface). These control links (channels) are transferred as messages between adjacent sectors to handover the train from one to the other. The sector receiving the channels takes over control of the train until the train leaves it, in which case a handover occurs. The network may of course be extended to contain more sectors, bifurcations, terminals and trains but, for simplicity, we just require one sector to control at most one train at a time. The status of a sector is hence either *free* or *busy*. All sectors are initially free. As Fig. 1 shows, if a sector is free (a), i.e., it handles no train, then it will keep on stand-by, monitoring requests from its neighbour sector(s). Once it receives a handover request, it switches to the busy status and gets ready to take control of the approaching train. If the sector is busy, on the contrary, it will reply to its neighbour to refuse the handover request. It will return to the free status once the train it controls leaves the current sector. As illustrated by (c) and (d) in Fig. 1, each sector has a handover point, near the end point of the sector. Once the train reaches the handover point of a sector, the sector will raise a handover request to the next sector. If the handover request is accepted (c), then it will transfer the channels connecting to the train to the next sector. Otherwise (d), it will decelerate the train to prevent it from entering the adjacent sector, and then try to hand it over again. The terminus can be regarded as a special sector that decelerates the train until it becomes immobile i.e., “terminates”.

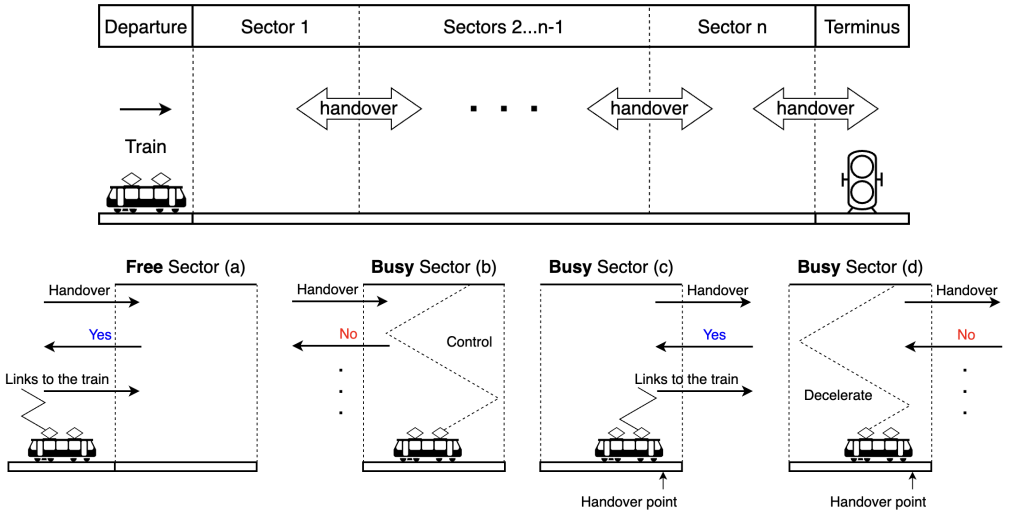


Fig. 1. The overall framework of the train control system.

7.2 Verification of the Handover Protocol Scenarios

This section focuses on the verification of the most critical component of the railway network’s control: the handover protocol. The specification of the complete model of the network is included in the full version of the paper [52].

Unlike the complete case study, which features dynamicity, mobility, orchestration, the network considered in the present scenario contains only one train *Train* and two sectors, *Left* and *Right*.

Its purpose is to verify the robustness of the handover protocol with respect to physical models and observations manifesting bounded physical disturbances and, nonetheless, prove the approximate bisimulation between an abstract, perfect model, and the realistic model with noise/disturbance.

The handover protocol is verified against its two possible scenarios: a successful handover, next, and a failed one. The train is placed at the starting point of the left sector initially. Under the control of the two sectors, the train will run to the right and finally terminate at the end of the right sector. By using replication, it can be extended to a network containing potentially infinite trains, controllers, terminus and sectors, which can be created and terminated dynamically.

7.2.1 The Train Model. Let u represent an arbitrary disturbance of the environment on the train and let constant $\text{len} = 5$ km denote the length of each sector. In the real world, however, the run of a train will be disturbed by many factors such as wind, drift, heat, friction and slope. All these disturbances are abstracted as an external and uncontrolled input u in the model. We assume that u is free to change dynamically within certain bounds $[-0.1, 0.1]$. The **Train** first creates three private variables (vp, v, a) to denote its position p , velocity v , and acceleration a of the train. These variables are sent out via *channels* by $\overline{\text{channels}}\langle p, v, a \rangle$. The sector receiving them takes control of the train. The train runs according to the set of ODEs specified in **Run**. The boundary condition $p < 2\text{len}$ means that the length of the whole journey of the train will be $2\text{len} = 10$ km long. The ready set (interface) $\{\bar{p}, \bar{v}, a\}$ indicates that the position (p) and velocity (v) of the train are sensors while the acceleration (a) can be actuated by the sector controlling the train. Once the train reaches the end of the right sector, it sends its last position (2len) out before termination.

$$\begin{aligned} \text{Train} &\triangleq (vp, v, a) (\overline{\text{channels}}\langle p, v, a \rangle. \text{Run} \parallel \text{Observer}) \\ \text{Run} &\triangleq \left\{ \begin{array}{l|l} 0 & \dot{p} = v \\ 0 & \dot{v} = a + u \ \& \ p < 2\text{len}, \{\bar{p}, \bar{v}, a\} \\ 0 & \dot{a} = 0 \end{array} \right\} . \bar{p}\langle 2\text{len} \rangle \\ \text{Observer} &\triangleq \{0 \mid \dot{x} = v \ \& \ p < 2\text{len}\} (x'). \{x' \mid \dot{x} = 0\} \end{aligned}$$

Process **Observer** monitors the evolution of the train and calculates its location using a free variable x by sensing the recorded velocity v of the train, defined in process **Run**, without actuating the dynamics of the train. Once the evolution of the train terminates, the physical variables p, v, a are no longer visible. We hence sustain x to its latest value once the train reaches the end of the right sector ($p = 2\text{len}$), as specified by the ODE $\dot{x} = 0$. The dynamic of the **Train** is not affected. To prove bisimulation, however, it is necessary to use the **Observer** to track the position of the train, because the evolution of p is private to the **Train** (variable p is under the scope of a restriction v). The variable x in the **Observer** is used to mimic or simulate the evolution of p in **Run**.

7.2.2 The Left Sector. The **Left** sector first takes control of the train by receiving the physical variables of the train, i.e., $\text{channels}(p, v, a)$, and then starts its control **Loop**. Let us highlight the constants for the handover point $p_h = 4$ km, the track length $\text{len} = 5$ km, and the update period of the sector's controller $d = 1$ s. At each iteration of the **Loop**, the sector first reads the current position of the train, i.e., $p(p_0)$. If the train has not reached the handover point ($p_0 < p_h$), the sector will continue to read the current velocity, i.e., $v(v_0)$, compute a safe acceleration $f(p_0, v_0, \text{len}, d)$ based on the sensed data, and then actuate the train with the new acceleration.

The function $f : \mathbb{R}^4 \rightarrow \mathbb{R}$ reflects the control strategy and will be specified in Section 7.2.5. After that, the sector waits for d time units before starting the next iteration of control, i.e., $\text{wait}(d).\text{ctrl}$.

However, if the train has crossed the handover point ($p_0 \geq p_h$), the sector will send a handover request to the right sector (handover) and wait for a reply. If the reply is *no*, i.e., the handover is refused, it will continue controlling the train to decelerate it. If the reply is *yes*, the left sector sends

the physical variables of the train to the right sector ($\overline{\text{switch}}\langle p, v, a \rangle$), switching the control of the train from the left to the right sectors.

$$\begin{aligned} \text{Left} &\triangleq \text{channels}(p, v, a). \text{Loop} \\ \text{Loop} &\triangleq \mu \text{ctrl}. p(p_0). \left(\begin{array}{l} \text{if } p_0 < p_h \text{ then} \\ \quad v(v_0). \bar{a}\langle f(p_0, v_0, \text{len}, d) \rangle. \overline{\text{wait}}(d). \overline{\text{ctrl}} \\ \text{else} \\ \quad \overline{\text{handover}}. \left(\begin{array}{l} \overline{\text{yes}}. \overline{\text{switch}}\langle p, v, a \rangle \\ + \\ \text{no}. v(v_0). \bar{a}\langle f(p_0, v_0, \text{len}, d) \rangle. \overline{\text{wait}}(d). \overline{\text{ctrl}} \end{array} \right) \end{array} \right) \end{aligned}$$

7.2.3 The Right Sector: Successful Handover. In this scenario, since the **Right** sector contains no train initially, it should first reply $\overline{\text{yes}}$ to the *handover* request of the left sector. Then, it takes the control of the coming train by $\text{switch}\langle p, v, a \rangle$ and starts the control **Loop'**. At each iteration of **Loop'**, the right sector checks whether the train has reached the endpoint. If not ($p_0 < 2\text{len}$), it drives the train periodically according to the algorithm specified by f ; otherwise, control terminates.

$$\begin{aligned} \text{Right} &\triangleq \text{handover}. \overline{\text{yes}}. \text{switch}\langle p, v, a \rangle. \text{Loop}' \\ \text{Loop}' &\triangleq \mu \text{ctrl}. p(p_0). \text{if } p_0 < 2\text{len} \text{ then } v(v_0). \bar{a}\langle f(p_0, v_0, 2\text{len}, d) \rangle. \overline{\text{wait}}(d). \overline{\text{ctrl}} \text{ else } 0 \end{aligned}$$

REMARK 4. The physical variables p , v , and a in **Run** depict the continuous evolution of the position, velocity, and acceleration of the **Train**. Meanwhile, they can be used as channels by sectors to control the train. In this sense, they represent the “control” of the train: any sector obtaining these variables takes control of the train. In particular, these variables can be exchanged between sectors, indicating the handover of control, and reflecting the “mobility” of the system. In addition, this example reflects the convention of the classic π -calculus: “names are the first-class citizen”.

7.2.4 The Right Sector: Failed Handover. In this scenario, the right sector always refuses the handover request from the left. The behaviour of the right sector can be modelled by

$$\text{Right}' \triangleq \mu x. \text{handover}. \overline{\text{no}}. \bar{x}$$

It replies $\overline{\text{no}}$ whenever it receives a *handover* request.

7.2.5 Control Algorithm. We mentioned in Section 7.2.2 that a sector controls a train by using a control command function $f : \mathbb{R}^4 \rightarrow \mathbb{R}$. Given a real-time position p_0 , velocity v_0 , end point p_e and period d of the sector, function f computes the acceleration a of the train. Let $v_{\max} = 40$ m/s be the maximum velocity of the train and $a_{\min} = -1$ m/s² the braking deceleration of the train. The control performed by f uses the Maximum Protection Curve [53] and is computed as follows:

$$V_{\lim}(p_0, p_e) = \begin{cases} v_{\max} & \text{if } p_e - p_0 \geq \frac{v_{\max}^2}{(-2a_{\min})} \\ \sqrt{-2a_{\min} \cdot (p_e - p_0)} & \text{if } 0 < p_e - p_0 < \frac{v_{\max}^2}{(-2a_{\min})} \end{cases}$$

If the end point is farther than the minimal safe distance $-v_{\max}^2/(2a_{\min})$ of the train, the upper limit velocity of the vehicle can be the maximum $v_{\max}$. If not, the velocity should not exceed $\sqrt{-2a_{\min} \cdot (p_e - p_0)}$ in order not to enter the next sector. At each iteration, the control algorithm predicts the position p' and velocity v' of the train at the next period based on the maximum acceleration $a_{\max} = 1$ m/s². Concretely, they can be computed by $v' = v_0 + a_{\max} \cdot d$ and $p' = p_0 + v_0 \cdot d + \frac{1}{2} \cdot a_{\max} \cdot d^2$, where $d > 0$ is the update period of the sector. If, at the next period, the velocity does not exceed the upper limit computed as above, i.e., $v' \leq V_{\lim}(p', p_e)$, then the maximum acceleration $a_{\max}$ is safe. If not, it continues to test if a constant velocity (no acceleration

or deceleration) is safe, i.e., $v_0 \leq V_{\text{lim}}(p_0 + v_0 \cdot d, p_e)$. If not, the train brake with the minimal deceleration a_{min} . The above control strategy can be summarised as:

$$f(p_0, v_0, p_e, d) = \begin{cases} a_{\text{max}} & \text{if } v' \leq V_{\text{lim}}(p', p_e) \\ 0 & \text{if } v_0 \leq V_{\text{lim}}(p_0 + v_0 \cdot d, p_e) \\ a_{\text{min}} & \text{otherwise} \end{cases}$$

7.2.6 Systems. The **System** of the successful scenario (Section 7.2.3) is modelled by the composition of the **Train** and **Left** and **Right** sectors, and they communicate with each other through local channels (of scope restricted by ν):

$$\text{System} \triangleq (\nu \text{ link, handover, switch, yes, no})(\text{Train} \parallel \text{Left} \parallel \text{Right})$$

Channel *link* connects the **Train** and the **Left** sector. The other channels (*handover*, *switch*, *yes*, and *no*) connect the **Left** and **Right** sectors. Similarly, the **System'** of the failed scenario (Section 7.2.4) is as follows:

$$\text{System}' \triangleq (\nu \text{ link, handover, switch, yes, no})(\text{Train} \parallel \text{Left} \parallel \text{Right}')$$

7.2.7 Approximate Bisimulations. In the successful scenario, the train runs into the right sector smoothly and finally stops at the end of the right sector. If there is no disturbance, the expected behavior of the train can be described by the following specification (**SPEC**):

$$\begin{aligned} \text{SPEC} &\triangleq (\nu p, v)((\text{Up.Stable.Down}) \parallel \text{Observer}) \\ \text{Up} &\triangleq \{0, 0 \mid \dot{p} = v, \dot{v} = 1 \ \& \ p < 0.8 \text{ km}\}(p_1, v_1) \\ \text{Stable} &\triangleq \{p_1, v_1 \mid \dot{p} = v, \dot{v} = 0 \ \& \ p < 9.2 \text{ km}\}(p_2, v_2) \\ \text{Down} &\triangleq \{p_2, v_2 \mid \dot{p} = v, \dot{v} = -1 \ \& \ p < 10 \text{ km}\} \end{aligned}$$

Process **Observer** is defined in Section 7.2.1. According to this specification, the evolution of the (undisturbed) train is composed of the following three stages:

Up The train speeds up from the initial state ($p = 0$ m and $v = 0$ m/s) with the maximum acceleration 1 m/s^2 for 0.8 km when it reaches the maximum velocity 40 m/s.

Stable Then, the train runs with constant velocity (40 m/s) until it is 0.8 km close to the end of the right sector ($10 - 0.8 = 9.2$ km).

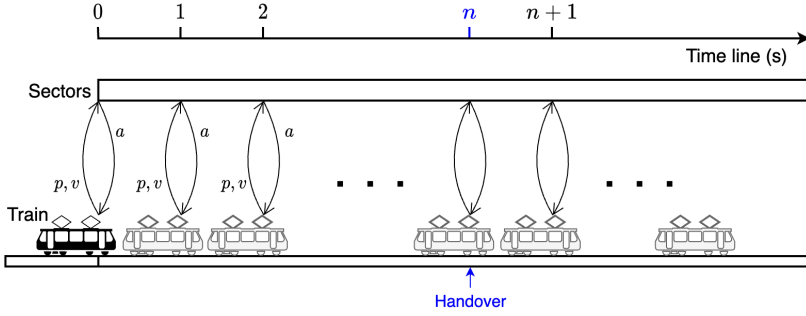
Down Finally, the train decelerates with the minimum acceleration -1 m/s^2 until it stops at the end of the right sector (10 km).

The specification (**SPEC**) describes the ideal model of the train control system without disturbance. The following proposition reveals that the disturbed **System** model (Section 7.2.6) is (0.4 km, 0)-approximately bisimilar to the **SPEC** model, indicating that the distance between the disturbed and undisturbed trains is within 0.4 km. By simulation, the maximum distance is around 0.205 km.

PROPOSITION 7.1. $\text{SPEC} \approx_{(0.4 \text{ km}, 0)} \text{System}$.

PROOF. We first prove that the distance between the real-time positions of the trains in **SPEC** (undisturbed) and **System** (disturbed) is not larger than 0.4 km, based on which we then prove that the following relation forms a (0.4 km, 0)-approximate bisimulation. More intuitively, it can be represented by the hybrid automaton in the left of Fig. 3, where each state only observes variable x .

The execution of the **System** model can be depicted by Fig. 2. The sector senses the position (p) and velocity (v), computes a new acceleration (a) based on the sensed data according to the algorithm specified in Section 7.2.5, and actuates the train with the computed acceleration every 1 second (the control period of the sector). Here the communication and computation are represented by a sequence of τ -actions, which are timeless. Once the **Left** sector senses that the train has reached the handover point, say at time n , it raises a handover request, which will be accepted by

Fig. 2. The execution of **System**.

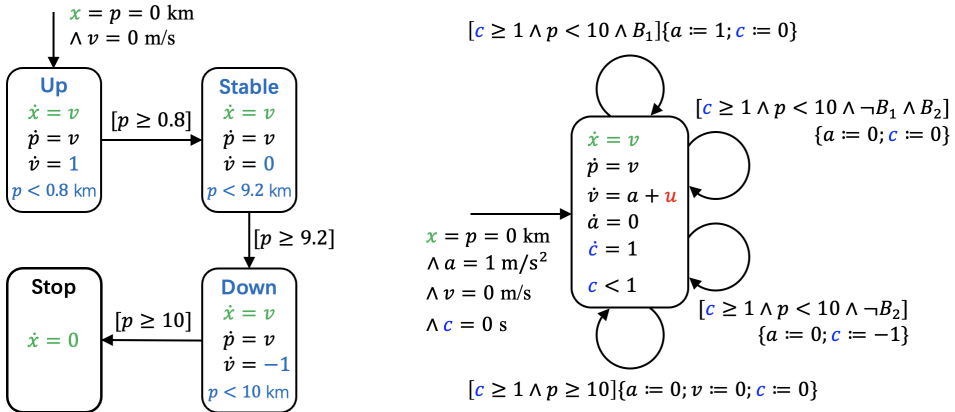
the **Right** sector, and then the right sector takes over the control. Since the handover is successful and both sectors adopt the same control algorithm (Section 7.2.5), the handover of the train is seamless and the train runs into the right sector without realising any difference.

$$\mathcal{S} = \{(\text{SPEC}, \text{System})\} \cup \left\{ (P, Q) \mid \begin{array}{l} \text{SPEC} \xrightarrow{\langle \rho_1, \emptyset \rangle} \dots \xrightarrow{\langle \rho_n, \emptyset \rangle} P \\ \text{System} \Rightarrow \dots \Rightarrow \langle \rho'_1, \emptyset \rangle \dots \Rightarrow \langle \rho'_m, \emptyset \rangle \Rightarrow Q \end{array} , |\rho_1 \frown \dots \frown \rho_n| = |\rho'_1 \frown \dots \frown \rho'_m| \right\}$$

The handover procedure can also be described by a sequence of τ -actions and hence it is timeless. After that, under the control of the right sector, the train runs and finally terminates at the end point of the sector. Intuitively, the execution of **System** can be depicted by the hybrid automaton on the right of Fig. 3, where

$$\begin{aligned} B_1 &\triangleq v + a_{\max} \cdot d \leq V_{\lim}(p + v \cdot d + \frac{1}{2} \cdot a_{\max} \cdot d^2, 2\text{len}) \\ B_2 &\triangleq v \leq V_{\lim}(p + v \cdot d, 2\text{len}) \end{aligned}$$

indicate that the acceleration 1 m/s^2 and the constant velocity are safe at the next period, respectively. We only observe the variable x of the state here (in red).

Fig. 3. The hybrid automata of **SPEC** (left) and **System** (right).

In order to observe the difference between the real-time positions (x) of the trains in **SPEC** and **System**, we compose the hybrid automata in Fig. 3, resulting in the composition shown in Fig. 4, where we subscript the variables of **SPEC** by 1 and **System** by 2 to distinguish between them.

Specifically, the composed hybrid automaton $\mathcal{H}$ consists of two independent components: one representing the evolution of **SPEC** variables (x_1, p_1, v_1, a_1, c_1) and the other representing **System** variables (x_2, p_2, v_2, a_2, c_2). Here, we introduce two fresh variables, a_1 and a_2 , representing acceleration, to unify the dynamics of both components. For example, the transition from **Up** to **Stable** in the **SPEC** is described by the reset operation $a_1 := 0$ in $\mathcal{H}$, when the guard condition $[p_1 \geq 0.8 \wedge c_1 \geq 1]$ is met. The clock c_1 , which is implicit in the **SPEC**, is introduced to force the transitions to take place at each time step. Consequently, proving the desired bisimulation relation is equivalent to showing that trajectories of the automaton $\mathcal{H}$ will never reach the unsafe region $|x_1 - x_2| \geq 0.4$ km. The latter problem is a typical *safety verification problem*: proving that a hybrid automaton, starting from an initial set of states, will never enter an unsafe region over an infinite-time horizon.

A well-established method to solve safety verification problem of hybrid systems is that of using *differential invariants*. Given a hybrid automaton, a differential invariant is a subset of the state space that remains invariant under the system's continuous dynamics, i.e., all trajectories starting from the differential invariant remain within it forever. To prove system safety, it suffices to find a differential invariant that includes the initial states and excludes the unsafe region. For the given problem, we can prove the existence of such a differential invariant that witnesses the safety of $\mathcal{H}$ with respect to the unsafe region $|x_1 - x_2| \geq 0.4$ km, implying that the relation $\mathcal{S}$ is a $(0.4 \text{ km}, 0)$ -approximate bisimulation, i.e., $\text{SPEC} \approx_{(0.4 \text{ km}, 0)} \text{System}$.

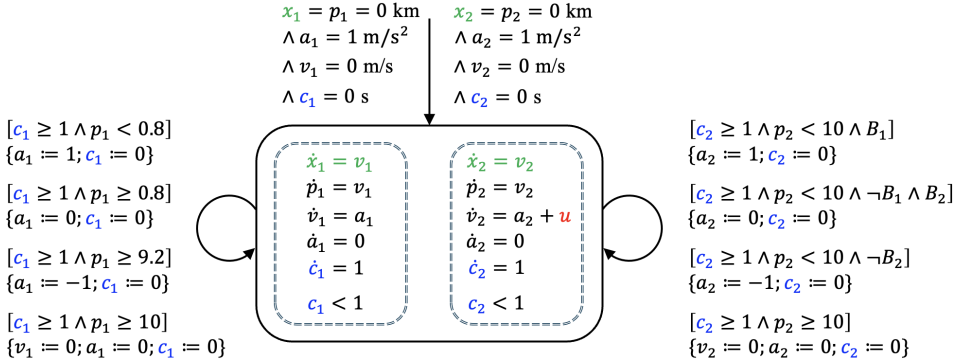


Fig. 4. The composed hybrid automaton $\mathcal{H}$.

The existence of the aforementioned differential invariant is established by using the method of *barrier certificates* [25, 41]. However, a detailed discussion of barrier certificates is beyond the scope (and size) of this paper. The full version of the paper [52] provides formal definitions of hybrid automata and barrier certificates, along with an explanation of how to compute the desired differential invariant using these techniques in the settings of our case study. $\square$

In the failed scenario, the expected behaviour of the undisturbed train can be described by the following specification (**SPEC'**), depicting the train stops at the end point of the left sector (5 km):

$$\begin{aligned}
 \text{SPEC}' &\triangleq (vp, v)((\text{Up}'.\text{Stable}'.\text{Down}')\|\text{Observer}) \\
 \text{Up}' &\triangleq \{0, 0 \mid \dot{p} = v, \dot{v} = 1 \ \& \ p < 0.8 \text{ km}\}(p_1, v_1) \\
 \text{Stable}' &\triangleq \{p_1, v_1 \mid \dot{p} = v, \dot{v} = 0 \ \& \ p < 4.2 \text{ km}\}(p_2, v_2) \\
 \text{Down}' &\triangleq \{p_2, v_2 \mid \dot{p} = v, \dot{v} = -1 \ \& \ p < 5 \text{ km}\}
 \end{aligned}$$

In the same (mirror) way as above, we can prove the following result:

PROPOSITION 7.2. $\text{SPEC}' \approx_{(0.3 \text{ km}, 0)} \text{System}'$.

8 Conclusions

In this article, we presented the definition of a compositional and expressive extension of the π -calculus with hybrid, discrete, real-timed and continuous processes, i.e., the hybrid π -calculus (**HpC**). The **HpC** allows to express and capture complex verification issues posed by dynamic, mobile and hybrid systems. We lifted the theory of bisimulation and congruence of the π -calculus to the **HpC**. In addition, we illustrate the expressive power, modelling and verification capabilities of the **HpC** by considering a functionally realistic model of a parametric hybrid (discrete/continuous) protocol: the handover protocol of a modular train control system.

As a result, we obtain the first characterisation of process calculus that has the capability to compositionally orchestrate the interaction of discretely timed processes and continuously timed ordinary differential equations (ODEs) in the presence of dynamicity and mobility. All items of the **HpC** are first-class citizens:

- Discrete and continuous processes are equal hybrid processes.
- Discrete channels and continuous variables are given a uniform treatment.
- All channels and processes, discrete or continuous, have both meanings in the discrete and a continuous operational semantics, providing the first hybrid semantics for the π calculus.

Last, but not least, the **HpC** supports control-theoretic approximate bisimulation in a process calculus with dynamic channels and processes.

Future Works

The **HpC** and its case study build the foundation to study the verification of modern, pervasive, internet(s) of hybrid things. Our future works focus on developing three aspects to empower its practicality:

- (1) Establish a proof system for the **HpC** based on the generalised hybrid Hoare logic [55];
- (2) Extend the **HpC** to a hybrid session π -calculus with a powerful session type system [31, 44];
- (3) Generate executable code automatically from **HpC** processes, the correctness of which (discretisation) will be proved using approximate bismulation (Definition 6.6).

Data-Availability Statement

The complete version of this article is available on arXiv [52]. It comprises the complete model of the train network subject to the case study in Section 7, the complete proof of Theorem 6.3, and the formal definition of barrier certificates used in the proof of approximate bisimulations of the handover scenarios (Section 7.2) along with a link to its implementation in [MATHEMATICA](#).

Acknowledgments

We wish to thank our anonymous reviewers and the program chairs for supporting our project. This work is funded by the National Key R&D Program of China under grant No. 2022YFA1005101, the Natural Science Foundation of China under grant No. 62402479, 62432005, 62192732, and the President's International Fellowship Initiative of the Chinese Academy of Sciences.

References

- [1] Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. 1992. Hybrid Automata: An Algorithmic Approach to the Specification and Verification of Hybrid Systems. In *Hybrid Systems (LNCS, Vol. 736)*, Robert L. Grossman, Anil Nerode, Anders P. Ravn, and Hans Rischel (Eds.). Springer, 209–229. https://doi.org/10.1007/3-540-57318-6_30
- [2] Bogdan Aman and Gabriel Ciobanu. 2013. Real-Time Migration Properties of rTiMo Verified in Uppaal. In *SEFM 2013 (LNCS, Vol. 8137)*, Robert M. Hierons, Mercedes G. Merayo, and Mario Bravetti (Eds.). Springer, 31–45. https://doi.org/10.1007/978-3-642-40561-7_3

- [3] Jos C. M. Baeten and Jan A. Bergstra. 1997. Process Algebra with Propositional Signals. *Theor. Comput. Sci.* 177, 2 (1997), 381–405. [https://doi.org/10.1016/S0304-3975\(96\)00253-8](https://doi.org/10.1016/S0304-3975(96)00253-8)
- [4] Jos C. M. Baeten and Cornelis A. Middelburg. 2002. *Process Algebra with Timing*. Springer. <https://doi.org/10.1007/978-3-662-04995-2>
- [5] Richard Banach. 2024. Core Hybrid Event-B III: Fundamentals of a reasoning framework. *Sci. Comput. Program.* 231 (2024), 103002. <https://doi.org/10.1016/J.SCICO.2023.103002>
- [6] Richard Banach, Michael J. Butler, Shengchao Qin, Nitika Verma, and Huibiao Zhu. 2015. Core Hybrid Event-B I: Single Hybrid Event-B machines. *Sci. Comput. Program.* 105 (2015), 92–123. <https://doi.org/10.1016/J.SCICO.2015.02.003>
- [7] Richard Banach, Michael J. Butler, Shengchao Qin, and Huibiao Zhu. 2017. Core Hybrid Event-B II: Multiple cooperating Hybrid Event-B machines. *Sci. Comput. Program.* 139 (2017), 1–35. <https://doi.org/10.1016/J.SCICO.2016.12.003>
- [8] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto L. Sangiovanni-Vincentelli, Werner Damm, Thomas A. Henzinger, and Kim G. Larsen. 2018. Contracts for System Design. *Found. Trends Electron. Des. Autom.* 12, 2-3 (2018), 124–400. <https://doi.org/10.1561/10000000053>
- [9] Jan A. Bergstra and Cornelis A. Middelburg. 2005. Process algebra for hybrid systems. *Theor. Comput. Sci.* 335, 2-3 (2005), 215–280. <https://doi.org/10.1016/J.TCS.2004.04.019>
- [10] Chiara Bodei, Pierpaolo Degano, Gian-Luigi Ferrari, and Letterio Galletta. 2016. Where Do Your IoT Ingredients Come From?. In *COORDINATION 2016 (LNCS, Vol. 9686)*, Alberto Lluch-Lafuente and José Proença (Eds.). Springer, 35–50. https://doi.org/10.1007/978-3-319-39519-7_3
- [11] Ningning Chen and Huibiao Zhu. 2024. A process calculus SMrCaIT for IoT. *J. Softw. Evol. Process.* 36, 5 (2024). <https://doi.org/10.1002/SMR.2595>
- [12] Xin Chen, Sriram Sankaranarayanan, and Erika Ábrahám. 2014. Under-approximate flowpipes for non-linear continuous systems. In *FMCAD 2014. IEEE*, 59–66. <https://doi.org/10.1109/FMCAD.2014.6987596>
- [13] Gabriel Ciobanu and Maciej Koutny. 2011. Timed Mobility in process algebra and Petri nets. *J. Log. Algebraic Methods Program.* 80, 7 (2011), 377–391. <https://doi.org/10.1016/J.JLAP.2011.05.002>
- [14] Pieter J. L. Cuijpers and Michel A. Reniers. 2005. Hybrid process algebra. *J. Log. Algebraic Methods Program.* 62, 2 (2005), 191–245. <https://doi.org/10.1016/J.JLAP.2004.02.001>
- [15] Guillaume Dupont, Yamine Ait Ameur, Neeraj Kumar Singh, and Marc Pantel. 2021. Event-B Hybridation: A Proof and Refinement-based Framework for Modelling Hybrid Systems. *ACM Trans. Embed. Comput. Syst.* 20, 4 (2021), 35:1–35:37. <https://doi.org/10.1145/3448270>
- [16] Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Rajarshi Ray, Olivier Lebeltel, Rodolfo Ripado, Antoine Girard, Thao Dang, and Oded Maler. 2011. SpaceEx: Scalable Verification of Hybrid Systems. In *CAV 2011 (LNCS, Vol. 6806)*, Ganesh Gopalakrishnan and Shaz Qadeer (Eds.). Springer, 379–395. https://doi.org/10.1007/978-3-642-22110-1_30
- [17] Ting Gan, Mingshuai Chen, Yangjia Li, Bican Xia, and Naijun Zhan. 2018. Reachability Analysis for Solvable Dynamical Systems. *IEEE Trans. Autom. Control.* 63, 7 (2018), 2003–2018. <https://doi.org/10.1109/TAC.2017.2763785>
- [18] Francesco Gavazzo. 2018. Quantitative Behavioural Reasoning for Higher-order Effectful Programs: Applicative Distances. In *LICS 2018*, Anuj Dawar and Erich Grädel (Eds.). ACM, 452–461. <https://doi.org/10.1145/3209108.3209149>
- [19] Antoine Girard and George J. Pappas. 2007. Approximation Metrics for Discrete and Continuous Systems. *IEEE Trans. Autom. Control.* 52, 5 (2007), 782–798. <https://doi.org/10.1109/TAC.2007.895849>
- [20] Dimitar P. Guelev, Shuling Wang, and Naijun Zhan. 2017. Compositional Hoare-Style Reasoning About Hybrid CSP in the Duration Calculus. In *SETTA 2017 (LNCS, Vol. 10606)*, Kim Guldstrand Larsen, Oleg Sokolsky, and Ji Wang (Eds.). Springer, 110–127. https://doi.org/10.1007/978-3-319-69483-2_7
- [21] Jifeng He. 1994. *From CSP to hybrid systems*. Prentice Hall International (UK) Ltd., GBR, 171–189.
- [22] Thomas A. Henzinger. 1996. The Theory of Hybrid Automata. In *LICS 1996. IEEE Computer Society*, 278–292. <https://doi.org/10.1109/LICS.1996.561342>
- [23] C. A. R. Hoare. 1978. Communicating Sequential Processes. *Commun. ACM* 21, 8 (1978), 666–677. <https://doi.org/10.1145/359576.359585>
- [24] A. Agung Julius, Alessandro D’Innocenzo, Maria Domenica Di Benedetto, and George J. Pappas. 2009. Approximate equivalence and synchronization of metric transition systems. *Syst. Control. Lett.* 58, 2 (2009), 94–101. <https://doi.org/10.1016/J.SYSCONLE.2008.09.001>
- [25] Hui Kong, Fei He, Xiaoyu Song, William N. N. Hung, and Ming Gu. 2013. Exponential-Condition-Based Barrier Certificate Generation for Safety Verification of Hybrid Systems. In *CAV 2013 (LNCS, Vol. 8044)*, Natasha Sharygina and Helmut Veith (Eds.). Springer, 242–257. https://doi.org/10.1007/978-3-642-39799-8_17
- [26] Gerardo Lafferriere, George J. Pappas, and Sergio Yovine. 2001. Symbolic Reachability Computation for Families of Linear Vector Fields. *J. Symb. Comput.* 32, 3 (2001), 231–253. <https://doi.org/10.1006/JSCO.2001.0472>
- [27] Ugo Dal Lago and Maurizio Murgia. 2023. Contextual Behavioural Metrics. In *CONCUR 2023 (LIPIcs, Vol. 279)*, Guillermo A. Pérez and Jean-François Raskin (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 38:1–38:17.

- <https://doi.org/10.4230/LIPICS.CONCUR.2023.38>
- [28] Ivan Lanese, Luca Bedogni, and Marco Di Felice. 2013. Internet of things: a process calculus approach. In *SAC 2013*, Sung Y. Shin and José Carlos Maldonado (Eds.). ACM, 1339–1346. <https://doi.org/10.1145/2480362.2480615>
 - [29] Ruggero Lanotte and Massimo Merro. 2018. A semantic theory of the Internet of Things. *Inf. Comput.* 259, 1 (2018), 72–101. <https://doi.org/10.1016/J.IC.2018.01.001>
 - [30] Jiang Liu, Jidong Lv, Zhao Quan, Naijun Zhan, Hengjun Zhao, Chaochen Zhou, and Liang Zou. 2010. A Calculus for Hybrid CSP. In *APLAS 2010 (LNCS, Vol. 6461)*, Kazunori Ueda (Ed.). Springer, 1–15. https://doi.org/10.1007/978-3-642-17164-2_1
 - [31] Rupak Majumdar, Nobuko Yoshida, and Damien Zufferey. 2020. Multiparty motion coordination: from choreographies to robotics programs. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 134:1–134:30. <https://doi.org/10.1145/3428202>
 - [32] MathWorks. 2013. *Simulink® User's Guide*. http://www.mathworks.com/help/pdf_doc/simulink/sl_using.pdf.
 - [33] MathWorks. 2013. *Stateflow® User's Guide*. http://www.mathworks.com/help/pdf_doc/stateflow/sf Ug.pdf.
 - [34] Robin Milner. 1993. The Polyadic π -Calculus: a Tutorial. In *Logic and Algebra of Specification*, Friedrich L. Bauer, Wilfried Brauer, and Helmut Schwichtenberg (Eds.). Springer, Berlin, Heidelberg, 203–246. https://doi.org/10.1007/978-3-642-58041-3_6
 - [35] Robin Milner. 1999. *Communicating and Mobile Systems: the π -Calculus*. Cambridge University Press.
 - [36] Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40. [https://doi.org/10.1016/0890-5401\(92\)90008-4](https://doi.org/10.1016/0890-5401(92)90008-4)
 - [37] Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, II. *Inf. Comput.* 100, 1 (1992), 41–77. [https://doi.org/10.1016/0890-5401\(92\)90009-5](https://doi.org/10.1016/0890-5401(92)90009-5)
 - [38] André Platzer. 2008. Differential Dynamic Logic for Hybrid Systems. *J. Autom. Reason.* 41, 2 (2008), 143–189. <https://doi.org/10.1007/S10817-008-9103-8>
 - [39] André Platzer. 2012. The Complete Proof Theory of Hybrid Systems. In *LICS 2012*. IEEE Computer Society, 541–550. <https://doi.org/10.1109/LICS.2012.64>
 - [40] André Platzer. 2018. *Logical Foundations of Cyber-Physical Systems*. Springer. <https://doi.org/10.1007/978-3-319-63588-0>
 - [41] Stephen Prajna and Ali Jadbabaie. 2004. Safety Verification of Hybrid Systems Using Barrier Certificates. In *HSCC 2004 (LNCS, Vol. 2993)*, Rajeev Alur and George J. Pappas (Eds.). Springer, 477–492. https://doi.org/10.1007/978-3-540-24743-2_32
 - [42] William C. Rounds. 2004. A Spatial Logic for the Hybrid π -Calculus. In *HSCC 2004 (LNCS, Vol. 2993)*, Rajeev Alur and George J. Pappas (Eds.). Springer, 508–522. https://doi.org/10.1007/978-3-540-24743-2_34
 - [43] William C. Rounds and Hosung Song. 2003. The Phi-Calculus: A Language for Distributed Control of Reconfigurable Embedded Systems. In *HSCC 2003 (LNCS, Vol. 2623)*, Oded Maler and Amir Pnueli (Eds.). Springer, 435–449. https://doi.org/10.1007/3-540-36580-X_32
 - [44] Alceste Scalas and Nobuko Yoshida. 2019. Less is more: multiparty session types revisited. *Proc. ACM Program. Lang.* 3, POPL (2019), 30:1–30:29. <https://doi.org/10.1145/3290343>
 - [45] D. A. van Beek, Ka L. Man, Michel A. Reniers, Jacobus E. Rooda, and Ramon R. H. Schiffelers. 2006. Syntax and consistent equation semantics of hybrid Chi. *J. Log. Algebraic Methods Program.* 68, 1-2 (2006), 129–210. <https://doi.org/10.1016/J.JLAP.2005.10.005>
 - [46] D. A. van Beek, Jacobus E. Rooda, Ramon R. H. Schiffelers, Ka L. Man, and Michel A. Reniers. 2005. Relating Hybrid Chi to Other Formalisms. In *IFM Doctoral Symposium 2005 (ENTCS, Vol. 191)*, Judi Romijn, Graeme Smith, and Jaco van de Pol (Eds.). Elsevier, 85–113. <https://doi.org/10.1016/J.ENTCS.2006.09.041>
 - [47] Peter C. W. van den Brand, Michel A. Reniers, and Pieter J. L. Cuijpers. 2006. Linearization of hybrid processes. *J. Log. Algebraic Methods Program.* 68, 1-2 (2006), 54–104. <https://doi.org/10.1016/J.JLAP.2005.10.003>
 - [48] Rob J. van Glabbeek and W. P. Weijland. 1996. Branching Time and Abstraction in Bisimulation Semantics. *J. ACM* 43, 3 (1996), 555–600. <https://doi.org/10.1145/233551.233556>
 - [49] Shuling Wang, Zekun Ji, Xiong Xu, Bohua Zhan, Qiang Gao, and Naijun Zhan. 2024. Formally Verified C Code Generation from Hybrid Communicating Sequential Processes. In *ICCPs 2024*. IEEE, 123–134. <https://doi.org/10.1109/ICCPs61052.2024.00018>
 - [50] Wanling Xie, Huibiao Zhu, and Xi Wu. 2023. A Timed Calculus with Mobility for Wireless Networks. In *CNIOT 2023*. ACM, 653–657. <https://doi.org/10.1145/3603781.3603896>
 - [51] Wanling Xie, Huibiao Zhu, and Qiwen Xu. 2021. A process calculus BigrTiMo of mobile systems and its formal semantics. *Formal Aspects Comput.* 33, 2 (2021), 207–249. <https://doi.org/10.1007/S00165-021-00530-X>
 - [52] Xiong Xu, Jean-Pierre Talpin, Shuling Wang, Hao Wu, Bohua Zhan, Xinxin Liu, and Naijun Zhan. 2025. HpC: A Calculus for Hybrid and Mobile Systems - Full Version. *CoRR* abs/2501.09430 (2025). <https://doi.org/10.48550/ARXIV.2501.09430> arXiv:2501.09430
 - [53] Xiong Xu, Shuling Wang, Bohua Zhan, Xiangyu Jin, Jean-Pierre Talpin, and Naijun Zhan. 2022. Unified graphical co-modeling, analysis and verification of cyber-physical systems by combining AADL and Simulink/Stateflow. *Theor.*

- Comput. Sci.* 903 (2022), 1–25. <https://doi.org/10.1016/J.TCS.2021.11.008>
- [54] Gaogao Yan, Li Jiao, Shuling Wang, Lingtai Wang, and Naijun Zhan. 2020. Automatically Generating SystemC Code from HCSP Formal Models. *ACM Trans. Softw. Eng. Methodol.* 29, 1 (2020), 4:1–4:39. <https://doi.org/10.1145/3360002>
- [55] Naijun Zhan, Bohua Zhan, Shuling Wang, Dimitar P. Guelev, and Xiangyu Jin. 2023. A Generalized Hybrid Hoare Logic. *CoRR* abs/2303.15020 (2023). <https://doi.org/10.48550/ARXIV.2303.15020> arXiv:2303.15020
- [56] Chaochen Zhou, Ji Wang, and Anders P. Ravn. 1995. A Formal Description of Hybrid Systems. In *Hybrid Systems III: Verification and Control (LNCS, Vol. 1066)*, Rajeev Alur, Thomas A. Henzinger, and Eduardo D. Sontag (Eds.). Springer, 511–530. <https://doi.org/10.1007/BFB0020972>

Received 2024-10-15; accepted 2025-02-18